

Developing Serverless Solutions on AWS

Amazon Web Services

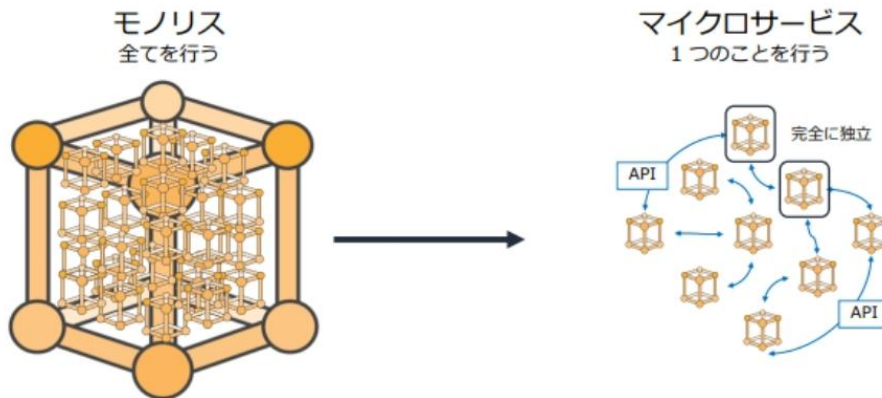
■ AGENDA

- 01 | サーバーレス思考
- 02 | API 駆動の開発と同期イベントソース
- 03 | 認証、認可、アクセスコントロールの概要
- 04 | サーバーレスデプロイのフレームワーク
- 05 | Amazon EventBridge と Amazon SNS を使用してコンポーネントを疎結合化する
- 06 | キューとストリームを使用したイベント駆動型開発
- 07 | 優れた Lambda 関数を書く
- 08 | Step Functionsを使用したオーケストレーション
- 09 | オブザーバビリティとモニタリング
- 10 | サーバーレスアプリケーションのセキュリティ
- 11 | サーバーレスアプリケーションでのスケール処理
- 12 | デプロイパイプラインの自動化

01 : サーバーレス思考

Chapter 1

■ モノリシック アーキテクチャとマイクロサービス アーキテクチャの比較



モノリシック アプリケーションとは何ですか？

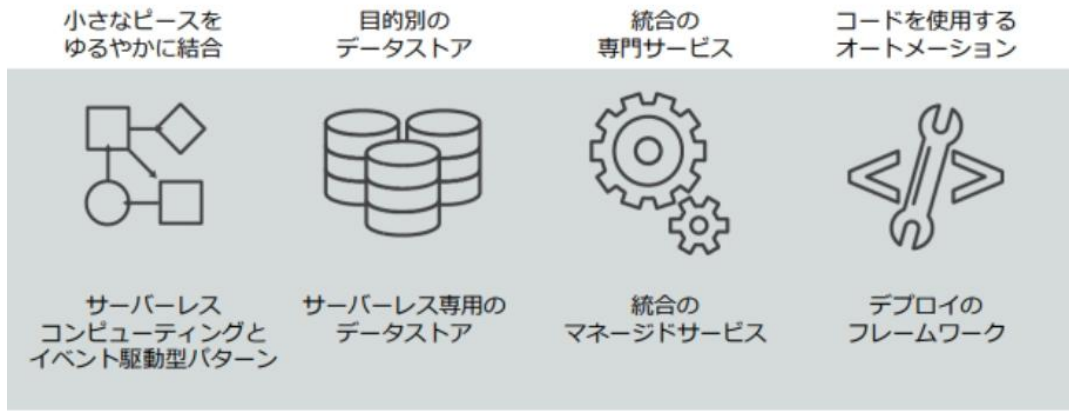
モノリス アプリケーションには、ユーザー インターフェイスとデータが単一のプラットフォームから単一のプログラムに結合される、密結合されたサービスがあります。これは自己完結型であり、他のコンピューティング アプリケーションから独立しています。

マイクロサービス アプリケーションとは何ですか？

マイクロサービス アプリケーションには次の特徴があります。

- 範囲が厳密に決められている
- 強力にカプセル化
- 疎結合
- 独立して展開可能
- スケーラブル

■ モダンなサーバーレスアプリケーションの構築



ビルダーの視点から見ると、仮想サーバーからコンテナ、そしてサーバーレスに至るクラウドへのインフラストラクチャの抽象化は、モノリシックアプリケーションを離れてサービス指向アーキテクチャとマイクロサービスに移行することと一致しています。

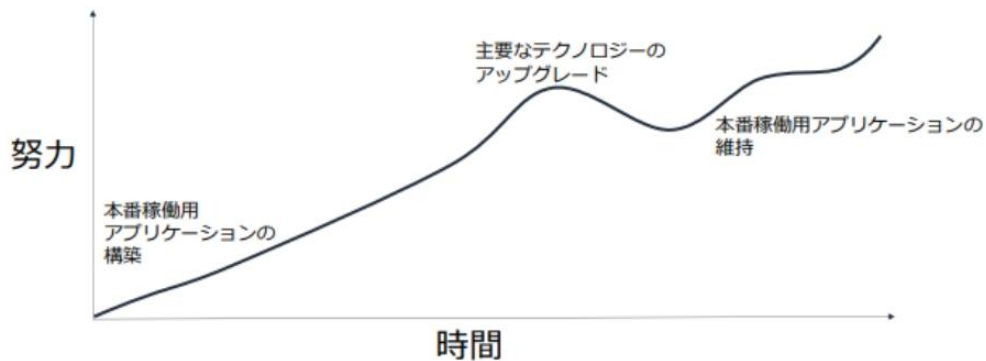
分散アプリケーションやマイクロサービスのアイデアは目新しいものではありませんが、クラウドサービスの進化により、次の原則に従ってアプリケーションを簡単に構築できるようになりました。

- ・小規模でゆるやかに結合されたピースと専用のデータストアを使用することで、柔軟性が向上し、システムの他の部分に影響を与えることなく変更を行い、各ピースを個別にスケールおよびデプロイし、可逆的な意思決定を組み込むことができます。
- ・専用の統合サービスを使用すると、配信速度を向上させ、サービス間の統合作業を軽減します。
- ・Infrastructure as Code を記述して、できるだけ多くのデプロイプロセスを自動化します。これにより、オーバーヘッドが最小限に抑えられ、信頼性が向上するため、環境のレプリケーションが簡単になり、頻繁にデプロイできるようになります。

サーバーレス は、モダンアプリケーションへの効率的なアプローチです。

- ・サーバーレスコンピューティングサービスを使用すると、ビジネスロジックの作成に集中でき、その間、他のユーザーはスケーリングや高可用性を処理することができます。
- ・専用のサーバーレスデータベースを使用すると、各サービスが必要とするデータのタイプとボリュームに合った機能と容量を選択できます。
- ・マネージドサービスは、API プロキシ、メッセージング、オーケストレーションを提供し、AWS Lambda およびその他のアマゾン ウェブ サービス (AWS) のサービスとの統合を事前に構築します。これにより、非同期のイベント駆動型パターンを使用してサービスを接続し、疎結合化と柔軟性を実現しながら、統合の構築で差別化されていないリフティングを軽減してコンポーネントを接続することができます。
- ・サーバーレスデプロイフレームワークにより、Infrastructure as Code をデプロイに使用し、デプロイパイプラインを自動化することが容易になります。

■ モノリスは当初は簡単でも後から難しくなる



モノリシックアプローチを離れてアプリケーションの構築に移行することは企業にとって決して容易なことではありません。

モノリシックアプリケーションでは、当初は簡単にスタートします。構築もテストもより簡単に行うことができます。また、デプロイするアーティファクトは 1 つだけです。それほど複雑でないアプリケーションでは、モノリシックアプローチが長期間にわたって最適なオプションになる場合があります。

時間の経過とともに成長する複雑なアプリケーションの場合は、コンポーネント間の相互依存性や開発チームが何らかのものをテストおよびデプロイするために必要となる調整が難しくなっていくため、実際に開発者の生産性が低下する可能性があります。

社内で開発する必要があるテクノロジースタックが 1 つあります。そのテクノロジーの開発者を雇って訓練する必要があり、他の言語のスキルを活用することはできません。

テクノロジースタックに何らかの変更があると、大きなプロジェクトになります。

データベースが 1 つあります。最も要求の厳しいユースケースの場合、コストと容量が増大します。データベースやデータ構造に適していない運用パフォーマンスは、パフォーマンスの向上のために調整され続けますが、いずれ限界に達し、再設計しなければそれ以上向上できなくなります。

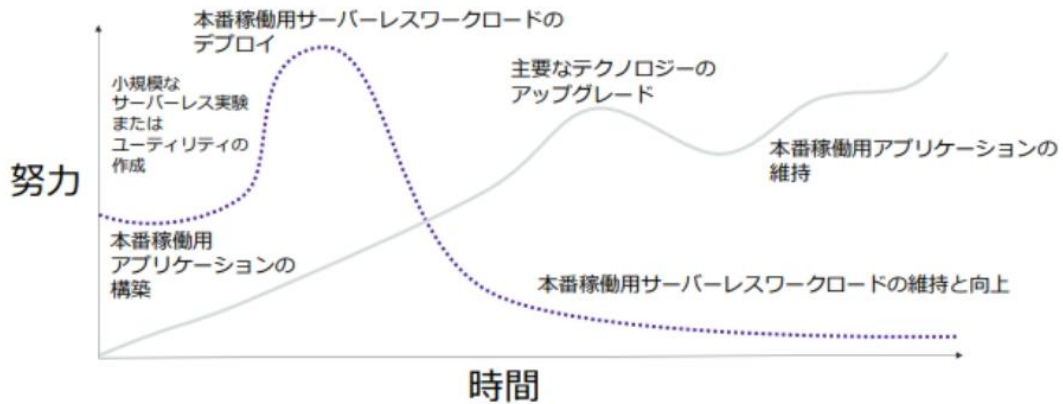
運用の観点から見ると、多くの開発者が共有リリースパイプラインを通じて変更をプッシュしており、ライフサイクルの多くのポイントでストレスが生じています。

- 新しい機能を利用するために共有ライブラリをアップグレードする場合は、他のすべてのユーザーが同時にアップグレードできるように調整する必要があります。
- 自分が担当している機能の重要な修正をすぐにプッシュしたい場合でも、他のすべてのユーザーの処理中の変更とマージする必要があります。このため、「マージフライデー」と呼ばれる状況、さらには「マージウィーク」と呼ばれる状況に陥り、すべての開発者が次のリリースのために変更をコンパイルして、競合を解決する必要があります。
- 開発が終了した後も、デリバリーパイプラインを介して変更をプッシュする時にオーバーヘッドが発生することがあります。アプリケーション全体を再構築し、すべてのテストスイートを実行して回帰の問題がないことを確認し、アプリケーション全体をもう一度デプロイする必要があります。

新しい機能を導入すると、調整、デプロイ、および継続的な運用作業がさらに複雑になります。技術的負債が積み重なり、コードを解くことはほぼ不可能になります。

このオーバーヘッドの例として、Amazon はかつて、マイクロサービスアプローチに移行する前に、モノリシックアプリケーションを本番環境にデプロイする役割を担った専門チームを配備していました。

■ モダンアプリケーションは当初は難しくても後から簡単になる



最新のサーバーレスアプリケーションを使用する場合、開始に要する労力が増大します。ここで説明したモダンアプリケーションの特性に移行するには、全員が間違えながら学習していくため、多大な労力が必要になります。多くの場合、必要としているアプローチを熟知している人はほとんどいません。

実験やユーティリティから本格的な本番稼働用アプリケーションに移行するには、テクノロジーと企業の変更を含む多大な労力を必要とします。その最初のハードルを乗り越えるためにはエネルギーが必要となるため、何らかの開始イベントや転換点がない限り、企業は立ち止まってしまう可能性があります。最初の学習曲線を過ぎると、このアプローチがもたらす俊敏性によって、新しい機能を開発するスピードや、それらを継続して実行するために取り組むスピードが加速し始めます。

- 個々のサービスを個別にスケールできます。
- サービスごとに最適なテクノロジスタックとデータストアを選択します。
- 必要な頻度で変更を個別にデプロイします。
- チームが特定のサービスに集中して、そのサービス内でイノベーションを実現できるようにします。

■ 比喩表現: フードトラックを着席形式のダイナーに拡張する

- ビジネスの成長をサポートするためのタスクの分割に影響を及ぼす要因は何ですか？



フードトラックでは1人ですべての業務をこなします。紙で注文を取り、現金のみの支払いを受け、注文ごとに食べ物を作り、在庫を管理します。グリル、フライヤー、小さなオープン、冷蔵庫が1つずつあり、それらすべてが、立っているところから手の届く範囲に配置されています

これまでに行った拡張では、繁忙日のサポートとして数人を雇い、全員が全てのことを少しずつ受け持っています。コミュニケーション方法は非常にインフォーマルです。うまく機能していますが、スケーラブルではありません。

より多くのお客様と製品ラインに対応できる環境に適応する必要があります。

■ 分散サービスとは、分散データを意味します

利点	課題
• 1つの万能データベースではなく、専用のデータストアを使用する • アクセスパターンとクエリパターン用に最適化する • データベースを個別にスケールする	• 結果整合性を受け入れる • 分散トランザクションと部分的な完了を処理する • 抽出、変換、ロード (ETL) の適切なソリューションを計画する

分散データの主な利点は、1つの万能データベースで妥協するのではなく、サービスに適した専用のデータストアを選択できることです。これにより、各データストアをアクセスパターンとクエリパターンに一致させ、そのパターンに合わせて最適化できます。また、最も要求の厳しいコンポーネントのキャパシティを購入するのではなく、データベースの選択肢を個別にスケールすることができます。

課題は、読み取りと書き込みを行う単独のデータベースがないことです。処理する分散ストアが多数あるので、必要に応じてデータを共有する方法を見つける必要があります。つまり、CAP 定理で説明されているように、結果整合性を受け入れる必要があるということです。

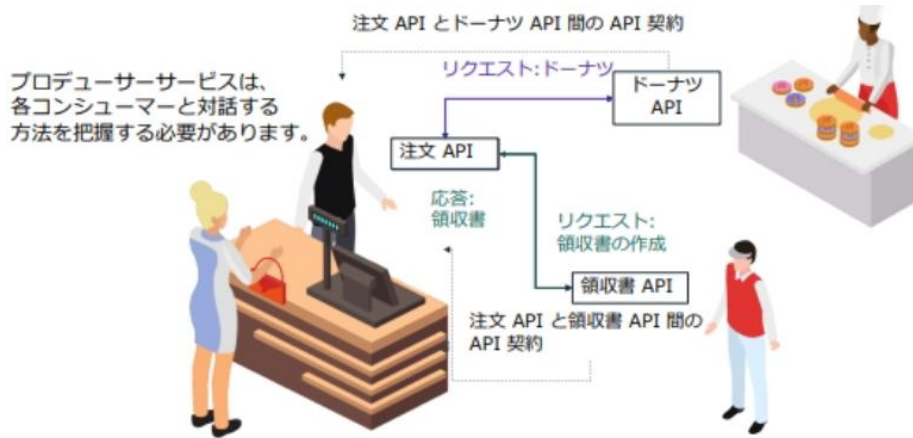
複数のデータストアを通過するトランザクションや、特に途中で失敗するトランザクションを処理するには、慣れない方法を使用しなければならないこともあります。また、1つのデータストアに書き込まれるデータを、それを必要とする他のコンポーネント（レポート作成に必要なトランザクションデータなど）と共有する方法も決定する必要があります。

■ イベント駆動型設計～指示されたコマンドを介してサービスが通信できる～



各コマンドを特定の受信者に明示的に発行し、応答を要求します。
この例では、カウンター店員またはマネージャーが注文を受け付け、レジ係に注文を記録して領収書を作成するように指示し、パン職人またはシェフには注文された食べ物を作るように指示します。

■ サービスは通常、API を使用してこれらのタイプのコマンドと通信できる



サービス間の通信は、同期または非同期にすることができます。

同期の例は次のとおりです。注文サービスが領収書サービスにリクエストを送信すると、領収書サービスはお客様への領収書を作成し、作成が完了するとすぐに領収書を返します。注文サービスは、領収書サービスが作業を完了し、応答を返すまで次に進みません。

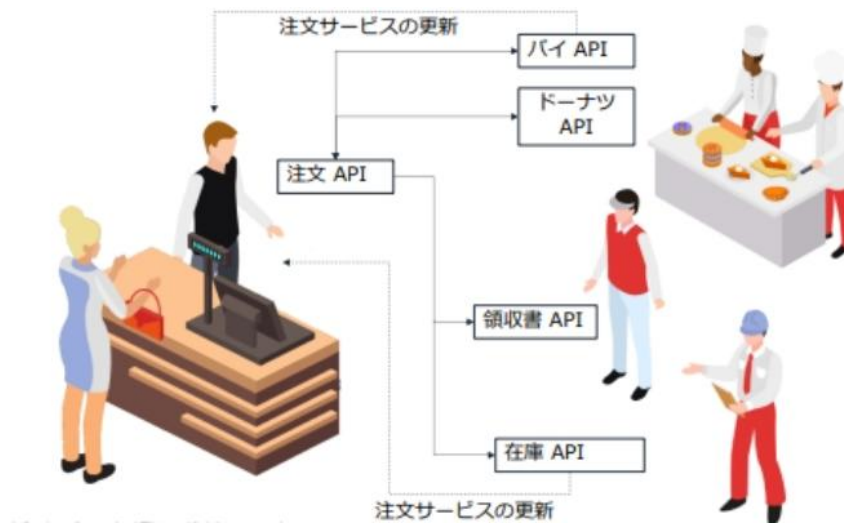
非同期の例は次のとおりです。注文サービスがドーナツ API からドーナツをリクエストしても、API は作ったドーナツを返しません。棚からドーナツを取り出すか、あるいは注文を満たすためのドーナツを作るというダウンストリームのステップがあります。注文サービスは、これらのステップを待つ必要はなく、次の注文に進みます。

注文サービスは、ドーナツサービスからすぐに確認応答を受け取ります。この確認応答には、注文プロセスで利用できる識別子が含まれており、この識別子によって、ドーナツの準備ができたことを確認できます。注文を完了する作業は非同期的に発生しているため、注文サービスが次の注文に進むことが妨げられることはありません。

API は、製品とそのサービスの消費者との間の契約として機能します。この契約によって、プロデューサーと消費者のコミュニケーション方法や、両者の間で渡される情報、消費者が期待どおりに応答しない場合の対処方法が決まります。

各消費者サービスに適切に構築された API がある場合でも、注文サービスは各消費者サービスと対話する責任を担います。

■ 新しいコンシューマーの追加とは、プロデューサーサービスを更新すること



新しいコンシューマーごとにプロデューサーサービスを更新し、新しい API の責任を処理する必要があります。プロデューサーサービスは、コンシューマー API のエンドポイントを認識し、使用すべき再試行動作を理解し、各コンシューマーサービスのソフトウェア開発キット (SDK) にバンドルする必要があります。これにより、データを消費する新しいサービスを追加するのに時間がかかり、サービスのビジネスロジックに直接関係しない注文サービスがさらに複雑になります。

API に変更が必要な場合は、注文サービスチームが更新を担当します。

■ イベント

- 状態の変更に関する情報をサービス間で共有するための主要なメカニズム
- 観察可能、指示なし
- 通常は非同期的に処理

イベント駆動型設計では、イベントは、サービス間で情報を共有するための主要なメカニズムです。イベントは指示されるのではなく観察可能です。イベント駆動型設計では、イベント処理は通常非同期で行われます。

■ サービスは観測可能なイベントを生成することによって通信できる



イベントベースのアプローチでは、プロデューサーが観測可能なイベント（オーダー情報など）を発行し、関心のあるコンシューマーがイベントに対して非同期的にアクションを実行します。プロデューサーは、イベントメッセージをコンシューマーがアクセスできる場所（既知のエンドポイントなど）に配置しますが、コンシューマーが実行する操作やアクションを実行するコンシューマーは指定しません。また、プロデューサーに戻る直接的なルートはありません。

■ イベントプロデューサーは、メッセージングサービスを使用して複数のコンシューマーとイベントを共有できる



この例では、建物の正面とキッチン間にキッチンマネージャーを配置することで、キッチンがフロントエンドとバックエンド間のメッセージングサービスとして機能します。メッセージングサービスは、適切なダウンストリームコンシューマーとのイベントの共有を処理します。これにより、プロデューサーとコンシューマーをさらに疎結合化することができます。一般的な例として、プロデューサーが既知のエンドポイントにメッセージをパブリッシュし、サブスクライバーがそのエンドポイントからメッセージを取得する Pub/Sub モデルがあります。

例えば、パイ、ドーナツ、領収書のサービスは、注文に関するメッセージをサブスクライブします。この疎結合化により、新しいコンシューマーの追加作業が簡単になります。この例では、プロデューサーに影響を与えることなく、インベントリサービスを新しいコンシューマーとして追加します。

■ ルーティングルールまたはフィルタリングルールを適用すると、無関係なイベントの送信と処理の負荷が軽減される



この例では、ルーティングまたはフィルタリングのルールをサポートするメッセージングサービスを使用することで、不要なトラフィックを減らし、無駄なメッセージをフィルターアウトするコンシューマーの作業を削減できるという利点が得られます。

例えば、「無料パイ」の注文をすべてのコンシューマーに送信し、ドーナツサービスと請求サービスに処理すべき作業がないことを識別させるのではなく、メッセージングサービス（キッチンマネージャー）はルールを使用して、パイイベントをパイサービスとインベントリサービスのみに送信します。このオプションを使用すると、メッセージングサービスが送信する必要があるメッセージの数が減るため、コンシューマーの作業負荷が軽減されます。

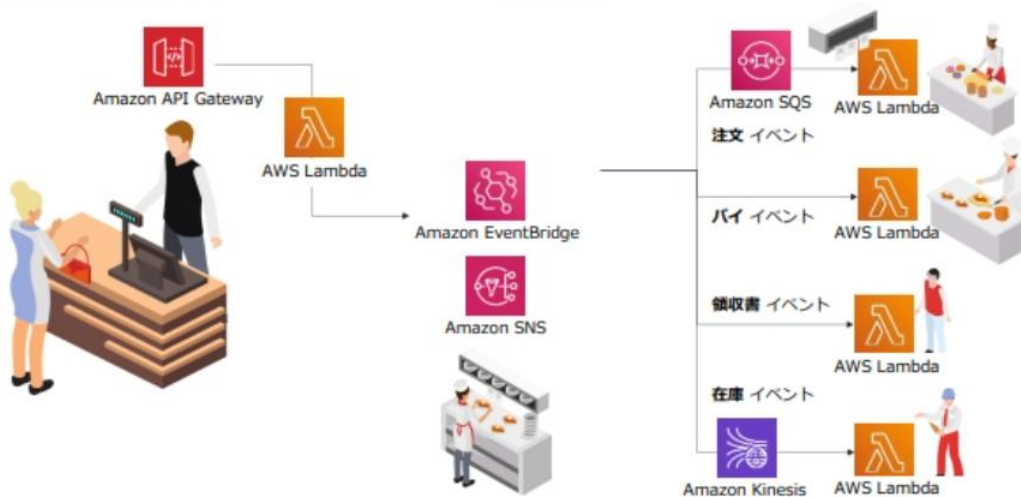
■ メッセージングサービスは、ダウンストリームのコンシューマーにバッファを提供することもできる



例えば、コンシューマーが独自のペースで受け取ることができるイベントを保存するキューまたはストリームがあります。

例えば、ドーナツサービスには、受信する注文イベントを収集し、それを制御されたバッチでドーナツメーカーに渡す独自の注文キューがあるとします。

■ AWS サーバーレスは最新のイベント駆動型設計に適している



AWS Lambda はサーバーレスコンピューティングを提供し、ここにビジネスロジックが存在します。

- Lambda 関数は、イベントソースに基づいて実行され、イベントオブジェクトを通じてデータを受信します。
- Lambda 関数は、他のサービスが消費できるイベントを生成します。
- Lambda では、AWS メッセージングおよびストリーミングサービスが生成するイベントがネイティブにサポートされます。これらの統合により、サービス間の通信を処理できるため、サービスの API の知識がなくても接続できます。

Amazon EventBridge および Amazon Simple Notification Service (Amazon SNS) は、メッセージングとファンアウトに使用できます。

- Amazon EventBridge は、コンシューマーからプロデューサーを抽象化し、ルールを使用してイベントをフィルターできるイベントルーターです。
- Amazon SNS は、サブスクライバーへの通知をフィルターできる Pub/Sub サービスです。
- どちらのサービスも Lambda の非同期イベントソースであり、Lambda ターゲットにすることができます。どちらにもエラーの処理機能が組み込まれています。

Amazon Simple Queue Service (Amazon SQS) と Amazon Kinesis Data Streams は、データのバッファリングに使用できます。

- Amazon SQS は、サービスが過負荷にならないように、リクエストをキューに追加することができ、イベントを保存するための耐久性に優れたストアとして機能します。
- Amazon Kinesis は、Lambda のストリーミングイベントソースとして機能します。一般的ユースケースは、ストリームにアップストリームイベントに関するデータが含まれている場合のデータ処理です。例えば、インベントリに影響を与えるイベントを Kinesis Data Streams に送信し、インベントリを分析して、午後に割引を動的に提供します。

Amazon API Gateway は同期イベントソースであり、アプリケーションへのフロントドアとして使用するのに最適です。次の場合は、同期接続が必要になる場合があります。

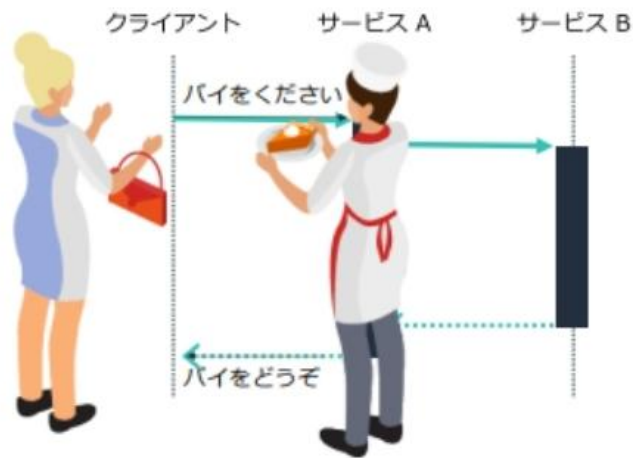
- クライアントが同期応答を必要としている。
- 応答方法を指示する必要がある。

- トランザクション境界が必要である。
- 絶対的な最低レイテンシーが必要である。
- メッセージは、単一のサービスの境界コンテキスト内でフローします。

02 : API 駆動の開発と 同期イベントソース

Chapter 2

同期コマンドはリクエスト/レスポンス



同期イベント処理

同期モデルでは、クライアントが要求を行い、応答を待機します。これは、作成、読み取り、更新、および削除 (CRUD) の API アクションに共通する比較的単純なやり取りです。

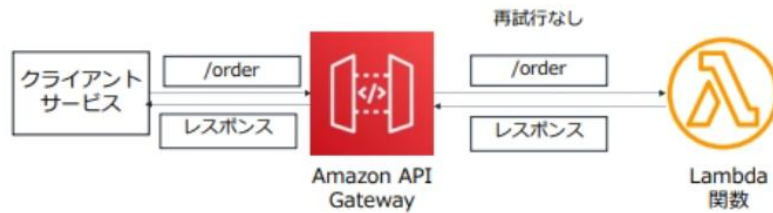
クライアントがサービス A に要求を行うと、サービス A が方向転換してサービス B を呼び出します。サービス A は、サービス B が必要な処理を完了するのを待ってから処理を続行し、最終的にサービス A に応答します。その後、この応答はクライアントに戻ります。

このアプローチの利点は、API コールへの即時応答です。API を使用して、要求と応答を制御することもできます。これらの例は、コンシューマが処理する前に要求の特性を検証したり、クライアントに渡す前に応答を再フォーマットしたりすることを示しています。

短所は、エラーまたは遅延ダウンストリームによってプロセス全体が停滞することです。サービス B で問題が起きると、エラーが発生したり、クライアントの待機時間が長くなったりします。

パン屋の例では、顧客がディスプレイケースからパイを購入することが同期イベントです。顧客は要求を行い、応答としてすぐにパイを受け取ってから、次の顧客がサービスを提供されます。クライアントは要求を行い、すぐに結果を受け取ります。

API Gateway が Lambda へのリクエストをプロキシし、関数を同期的に呼び出す



同期呼び出しでは、Lambda サービスは関数に直接イベントを送信し、呼び出し元に応答を返します。再試行は組み込まれていないため、要求するイベントは応答を待つ必要があります。つまり、リクエストを行っているプロデューサーサービスまたはクライアントにエラー処理を書き込むのは開発者の仕事です。AWS CLI と同期して関数を呼び出すには、requestResponse の呼び出しタイプを指定して invoke コマンドを使用します。

AWS コマンドラインインターフェイス (CLI) および AWS ソフトウェア開発キット (SDK) を使用して要求を行う場合は、クライアントタイムアウト、スロットリング、サービスエラーの再試行の組み込みオプションを使用します。

API Gateway は、Lambda との機能豊富な統合を提供します。

API Gateway は Lambda 関数を同期的に呼び出すことができ、30 秒のタイムアウトが組み込まれています。Lambda サービスが 30 秒以内に関数を正常に呼び出すことができない場合、または関数が実行されたがエラーが生成された場合、

API と呼び出し元のクライアントにエラーが返されます。

■ API Gateway により、API の構築とメンテナンスの複雑さを軽減

開発者向け機能	セキュリティ機能と拡張機能
• RESTful API または WebSocket API の作成 • 複数のバージョンの API をホストする • API 用の SDK の生成 • Swagger または OpenAPI 3.0 定義の Import/Export • 他の AWS のサービスおよびバックエンドとの統合 • プロキシ統合またはカスタム統合の作成	• 独自の署名バージョン 4 認可を記述せずに Lambda を呼び出す • 予想されるアクセスパターンに最も適したエンドポイントタイプの選択 • API へのアクセスの許可 • リクエストのスロットリング • カナリアデプロイメントの実行 • AWS WAF との統合

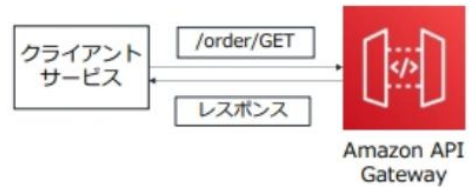
フルマネージドサービスである API Gateway を利用すれば、デベロッパーは規模にかかわらず、API の作成、公開、保守、モニタリング、保護を簡単に行えます。これにより、実装に伴う運用上の負担から解放され、信頼性の高いネットワーク保護が可能になり、認可の決定がポリシー内に一元化されるため、バグやコードに関する懸念を最小限に抑えることができます。

API Gateway が提供するセキュリティ上の主な利点は、AWS Identity and Access Management (IAM) 署名がクライアントに代わって呼び出しを行うことです。これがない場合、Lambda (または任意の AWS サービス) への直接呼び出しを行うには、署名バージョン 4 の認可を記述する必要があります。

■ RESTful API: ステートレスリクエスト/レスポンス

■ 特徴

- ◆ リクエスト/レスポンス
- ◆ HTTP メソッド (GET、POST など)
- ◆ 存続時間の短い通信
- ◆ ステートレス



■ API Gateway の 2 つのタイプ

- ◆ HTTP: 低レイテンシー、コスト効率の高い AWS 統合の設計
- ◆ REST: 現在より多くの機能を備えた旧世代の API
- ◆ HTTP API と REST API 間で選択する

前述のように、API Gateway は、RESTful API と WebSocket API をサポートしています。RESTful API は、標準の HTTP メソッドを使用するステートレスなリクエスト/レスポンス API を提供します。

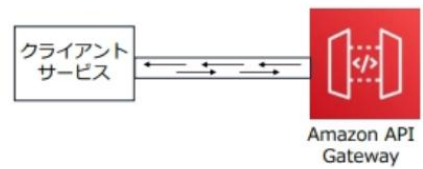
API Gateway では、RESTful API の 2 つのバージョン (REST または HTTP) から選択できます。HTTP API は、AWS Lambda を含む AWS サービスや HTTP エンドポイントとの低レイテンシーでコスト効率の良い統合のために設計されています。

REST API は、API リクエストとレスポンスを完全に制御でき、HTTP API にはない機能もあります。

■ WebSocket API : リアルタイム双方向通信

■ 特徴

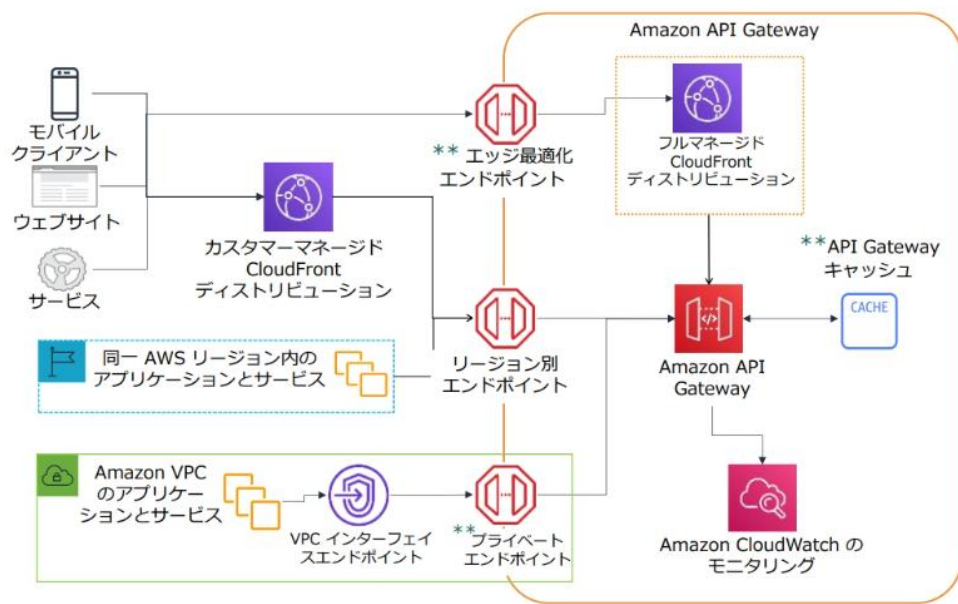
- ◆ 双方向通信チャンネル
- ◆ 存続時間の長い通信
- ◆ ステートフル



WebSocket API を使用すると、チャットアプリケーション、ストリーミングダッシュボードといったリアルタイム双方向通信アプリケーションを構築できます。

API Gateway は、バックエンドサービスとクライアント間のメッセージ転送を処理するために永続的な接続を維持します。

■ エンドポイントの種類



API Gateway は、API コールの受け入れと処理に関するすべてのタスクを処理します。すべての API タイプ (REST、HTTP、および WebSocket) が、リージョンのエンドポイントをサポートしています。リージョンのエンドポイントは、一般的なユースケースに推奨され、同じ AWS リージョンにあるクライアントの API を構築するために設計されています。

API Gateway では、すべての API タイプについて、Amazon CloudWatch を介して設定可能なロギングとメトリックスも提供します。

図に示すように、リージョンのエンドポイントの前に Amazon CloudFront ディストリビューションをセットアップするのがベストプラクティスです。これらのエンドポイントは、分散型サービス妨害 (DDoS) 攻撃から保護し、要求と応答のラウンドトリップ時間を短縮するために、Web サイトやその他の外部に重点を置いたサービスによってアクセスされます。

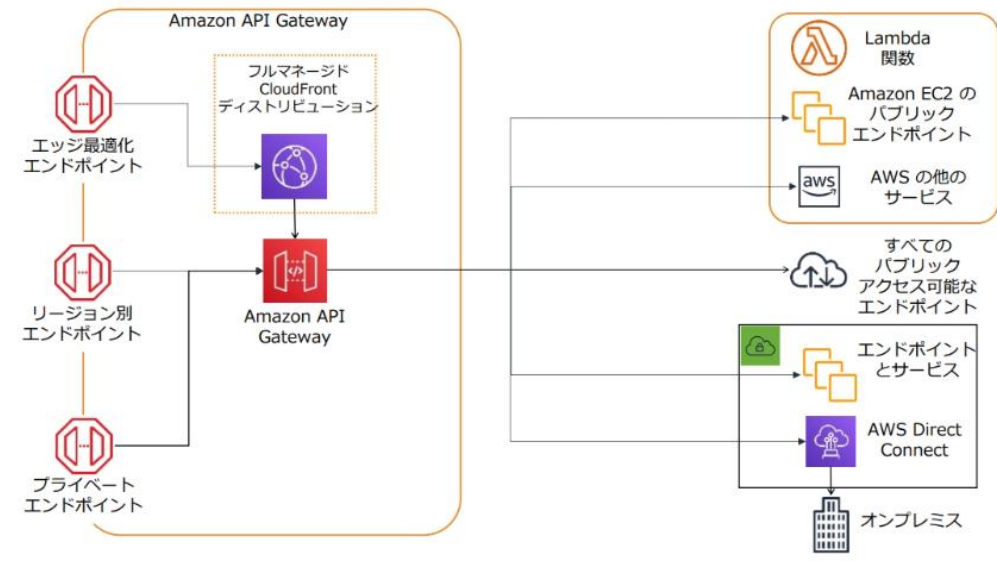
REST API には、バックエンドの負荷を削減し、定期的なリクエストを処理する際に発生するレイテンシーを軽減するためのオプションのキャッシュもあります。

REST API では、エッジ最適化エンドポイントのオプションもあります。このエンドポイントタイプでは、API Gateway は独自の CloudFront ディストリビューションを使用して、要求と応答のラウンドトリップ時間を短縮します。このエンドポイントタイプは、グローバルに分散されたクライアント向けに設計されており、

CloudFront ディストリビューションを介して組み込みの DDoS 保護を提供します。個別に CloudFront ディストリビューションをセットアップする必要はありません。このオプションは、HTTP API または WebSocket API では使用できません。

REST API には、Amazon Virtual Private Cloud (VPC) 内でのみアクセスできるプライベート API のオプションも用意されています。このエンドポイントタイプは、内部またはプライベートマイクロサービスによって使用される API を構築するために設計されています。

バックエンド統合



API に選択したエンドポイントのタイプに関係なく、API Gateway をサービスのプロキシとして使用して、Lambda やその他の AWS サービスと直接統合することができます。REST API と HTTP API の両方を使用して、Amazon VPC 内のエンドポイントに到達することもできます。

GraphQL は、アプリがサーバーからデータを要求（フェッチ）できるように設計されている

■ GraphQL スキーマ

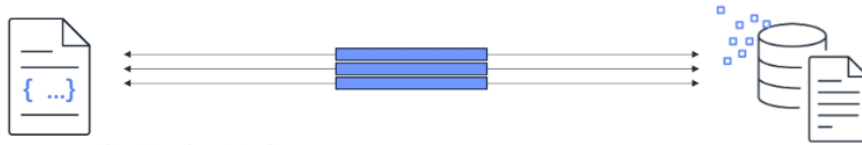
- ◆ API および実行可能なオペレーションも通過するデータの形状を定義する

■ リゾルバー

- ◆ タイプのスキーマのフィールドをデータソースに接続する。リゾルバーはリクエストを実行するためのメカニズムである

■ データソース

- ◆ GraphQL API がやり取りできる AWS アカウントのリソース



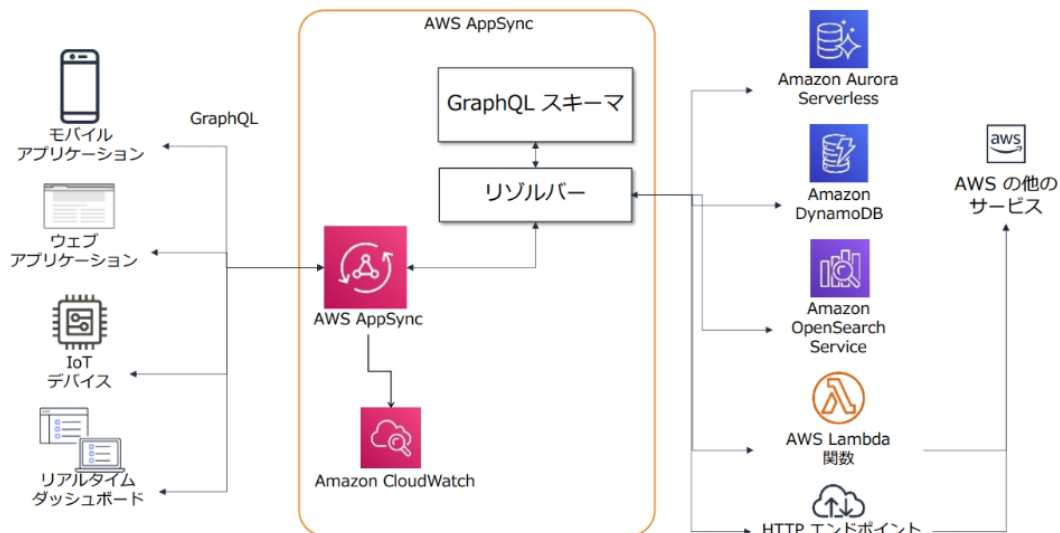
- クエリ: 読み取り専用の取得
- ミューテーション: 書き込み、必要に応じて更新されたデータを取得
- サブスクリプション: データを受け取るための、存続期間の長い接続

GraphQL は、アプリがサーバーからデータを要求（フェッチ）できるように設計されています。GraphQL スキーマは、データと実行可能なオペレーションを定義します。次の 3 つのトップレベルオペレーションがあります。

- クエリ: 読み取り専用の取得
- ミューテーション: 書き込み、必要に応じて更新されたデータを取得します。 ミューテーションにより、書き込み後にサーバー側のデータを返すことができるため、追加の呼び出しやネットワークのラウンドトリップが回避されます。
- サブスクリプション: データを受け取るための、存続期間の長い接続

GraphQL を使用すると、API は複数のデータソースとやり取りでき、リゾルバーはスキーマフィールドとデータソース間の接続を提供します。

仕組み



GraphQL API は、リソースと HTTP 動詞を中心に構造化されていません。代わりに、データ型を定義し、オペレーションによってリゾルバ経由でデータを取得する場所を設定します。

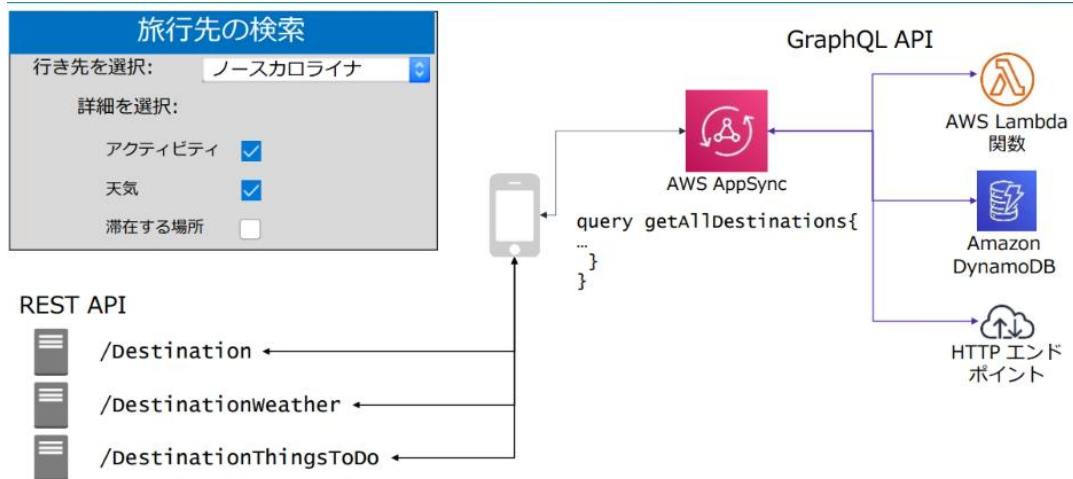
オペレーションは、クエリまたはミューテーションのいずれかです。クエリでデータを取得し、データを修正するときにミューテーションが使用されます。

AWS AppSync では、スキーマからリゾルバーを自動的に作成して接続したり、スキーマを作成して既存のテーブルからリゾルバーを接続したりできます。

データソースは、GraphQL API がやり取りできる AWS アカウントのリソースです。

AWS AppSync API は、複数のデータソースを操作するように設定できます。これにより、単一の場所にデータを集約できます。AWS AppSync は、すでに存在するアカウントから AWS リソースを使用する、またはスキーマ定義から、ユーザーに代わって Amazon DynamoDB テーブルをプロビジョニングできます。

GraphQL は 1 回のコールでデータを取得できる REST は複数回のコールでデータを取得する



REST API では、必要なデータを取得するために、複数のシーケンシャルコールを行うことが必要になる場合があります。この例では、ユーザーが目的地を選択し、目的地に関する詳細を取得できる旅行サイトがあり、旅行先の決定に役立てます。

通常、REST では、返される各種の情報は、呼び出す必要がある独自のリソースによって処理されます。この要求を満たすには、3 つの異なるコールが必要です。

1 つ目のコールでは選択した目的地に関する詳細を取得し、2 つ目のコールでは目的地の天気を取得し、3 つ目のコールでは、目的地に関連付けられた観光リストを取得します。

GraphQL では、`GetAllDestinations` のスキーマに、目的地、アクティビティ、天気、ホテルのフィールドが含まれます。1 回のコールで、必要なフィールド（この例では、特定の目的地であるノースカロライナ州のアクティビティと天気）のみを要求できます。リゾルバーは、その情報を提供するさまざまなデータソースに接続し、

AWS AppSync は結合された結果をクライアントに提供します。

ワークロードとチームにとって理にかなった API アプローチを選択

GraphQL と AWS AppSync	REST と API Gateway
- 強く型付けされたスキーマ /graphql への HTTP GET と POST - 同じ API で複数の認証オプション - クライアント固有の応答が必要 - 新しいアプローチのため、使用する既存のクライアントが少ない	- 強く型付けされたスキーマがない - 単一の /resource ですべての HTTP メソッドに対応 - リソースごとに 1 つの認証オプション - サーバー制御の応答 - Web サイト上の JavaScript も含む、多くのクライアントが REST に対応

GraphQL では、スキーマとデータ型を事前に定義する必要があります。スキーマ内にない場合は、クエリを実行できません。開発者は、スキーマをダウンロードして、スキーマからソースコードを生成して作業することができます。API を呼び出すとき、開発者はすべてのリクエストに対して HTTP GET または POST を

/graphql に使用し、必要なものについてのみクエリの詳細を変更できます。

REST を使用すると、データ構造を作成して要求されているものを検証することができますが、コードを記述するために必要なのは、API ドキュメントだけです。モデルを理解し、ドキュメントに基づいて API を記述し、各メソッドまたはリソースを呼び出す必要があります。

AWS AppSync では、同じ API で複数の認証オプションを使用できますが、API

Gateway では、リソースごとに 1 つの認証オプションのみ関連付けることができます。

多くのバックエンドソースからのデータを必要とするクライアント固有の応答が必要なアプリケーションには、GraphQL を検討してください。

既存のクライアントと統合する必要がある場合は、REST の方がはるかに成熟しており、使用するツールも豊富に揃っています。ほとんどのクライアントは REST 用に記述されています。

03 : 認証、認可、 アクセスコントロールの概要

Chapter 3

■ 同じものとして考えがちですが、はっきり区別することが重要です

認証 (AuthN)

- 本人であることを確認する
- アイデンティティを検証する

認可 (AuthZ)

- これから実行する操作に関する権限があるかどうかを判断する



認証 (AuthN) は、ユーザー本人であることの確認です。

認可 (AuthZ) は、ユーザーにどのような操作が許可されているかの確認です。

■ アイデンティティプロバイダーは、認証のための ID 情報を管理します



ユーザーのアイデンティティまたはその他のプリンシパルの作成と維持管理の責任を担います



分散ネットワーク内の他のアプリケーションに対して、ユーザーの認証を可能にします



フェデレーティッドログインでの信頼されたソースとして機能し、ユーザーはそのシステムの認証情報なしでシステムにアクセスできます

アイデンティティプロバイダー (IdP) は、システム内のユーザーのアイデンティティに関する信頼できる唯一のソースです。また、企業内のあらゆるアプリケーションを対象としたシングルサインオンや、フェデレーティッドログイン (Amazon、Facebook、Google ID を使用して別のアプリケーションにサインインするなど) を容易にします。

API Gateway を通じて API へのアクセスを認可するための一般的な 3 つの方法



JWT ベースの認可

- トークンの検証に基づいてリクエストを許可または拒否する
 - HTTP: JWT オーソライザー
 - REST: Amazon Cognito ユーザープール
- OIDC 規格準拠の **JWT** を使用



AWS IAM アクセス許可

- **IAM 認証情報**を持つリクエストが必要
- IAM ポリシーを API に適用
 - HTTP: ルートレベル
 - REST: API またはメソッドレベル
- オーソライザーによるリクエストの方法は**AWS API と同様**



Lambda オーソライザー

- Lambda 関数による**カスタム認可の実行が可能**
- Lambda によって返されたポリシーに基づいてリクエストを許可または拒否する
- ベアラートークン (SAML など) を使用するカスタムスキームをサポート

Open ID Connect (OIDC、<https://openid.net/connect/>) は、OAuth 2.0 プロトコル上のシンプルなアイデンティティレイヤーです。デベロッパーは、多くの場合、アイデンティティの管理を必要としない、フェデレーティッドアイデンティティ対応のウェブアプリやモバイルアプリで Open ID Connect を使用します。Open ID Connect を使用することで、クライアントは、認可サーバが実行する認証に基づいてユーザーのアイデンティティを検証し、REST に似た相互運用可能な方法でユーザーの基本的なプロフィール情報を取得できます。Open ID Connect は、2014 年にリリースされました。

OIDC は、オープン規格である JSON ウェブトークン (JWT) を使って、2 者間で安全に情報を送信します。Amazon API Gateway は、HTTP API と REST API の両方で JWT ベースの認可を可能にします。HTTP API は JWT オーソライザーを使用でき、REST API は Amazon Cognito ユーザープール (JWT オーソライザーを使用する) を使用できます。JWT オーソライザーでは、API Gateway がクライアントから API リクエストを使って送信される JWT を検証します。

Amazon Cognito ユーザープールでは、ユーザーはユーザープールに照らして認証され、OIDC トークンを取得します。API コールは、必要な JWT トークンが供給され、それが有効な場合にのみ成功します。API Gateway は、トークンの検証、およびオプションでトークン内のスコープに基づいてリクエストを許可または拒否します。API の各ルートに個別のオーソライザーを設定することも、複数のルートに同じオーソライザーを使用することもできます。

AWS Identity and Access Management (IAM) に基づくアクセス許可では、リクエストが IAM 認証情報を保有している必要があります。キー情報が認可ヘッダーに追加され、API Gateway が IAM ユーザーまたはロールに API の呼び出しが許可されるかどうかを判断します。

Lambda オーソライザーでは、Lambda 関数によってカスタム認可が実行されます。

Lambda オーソライザーが設定されている場合、API Gateway はそれを使用して API を呼び出します。API Gateway が認可情報を提供し、Lambda 関数がリクエストを許可または拒否するポリシーを返します。Lambda オーソライザーは、OAuth や Security Assertion Markup Language (SAML) などのベアラートークン認証戦略を使用するカスタム認可スキームや、発信者のアイデンティティを判断するためにリクエストパラメータを使用するス

キームを実装する場合に便利です。

このコースでは SAML ([http://docs.oasis-](http://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0-cd-02.html)

[open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0-cd-02.html](http://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0-cd-02.html)) は使用しませんが、企業や団体では、ID の管理が望まれるまたは必要とされる場合に、あらゆるシステムを対象としたエンタープライズシングルサインオンおよびフェデレーションで SAML が使用されています。SAML は、オンラインビジネスパートナー間でセキュリティ情報を記述および交換するための XML ベースのフレームワークを定義し、セキュリティドメインの境界をまたいで動作するアプリケーションが信頼できるポータブルな SAML アサーションの形式でセキュリティ情報を表現します。SAML は、2002 年にリリースされました。

■ Amazon Cognito はユーザーの認証と認可の複雑さを軽減します

開発者向け機能	セキュリティ機能と拡張機能
• 規格 (OIDC および OAuth 2.0) に基づく認証 • カスタマイズ可能な UI を使ったサインインとサインアップ • サードパーティーのソーシャルまたはエンタープライズ ID プロバイダーを通じたフェデレーティッドサインイン • JWT オーソライザーの IdP として設定可能	• 数億人規模のユーザーまで拡張可能なフルマネージドサービス • 多要素認証をサポート • リスクベースのアダプティブ認証による高度なセキュリティ • 漏えいした認証情報の使用を防止

■ AuthN および AuthZ に関する Amazon Cognito ユーザープールとフェデレーティッドアイデンティティの比較

コンポーネント	機能	説明
ユーザープール	認証	プロフィール管理機能を備えたアイデンティティプロバイダーとして機能する サードパーティー IdP とのフェデレーションによる認証のオプションを可能にする
フェデレーティッドアイデンティティ (ID プール)	認可	IAM ロールを使用して一時的な認証情報を生成し、AWS リソースにアクセス許可を付与する

Amazon Cognito には、さまざまな面から認証と認可を可能にする 2 つの機能があります。これらの機能の役割をはっきり区別することが重要です。

Amazon Cognito ユーザープールは IdP として機能します。ユーザープールを使用すると、ユーザーはウェブアプリケーションまたはモバイルアプリケーションにサインインできるのに加え、サードパーティー IdP とのフェデレーションによる認証を行うというオプションも可能となります。

認可の観点から見ると、ユーザープールは認可に使用できる JWT を発行します。これらのトークンを検証できるものは次のとおりです。

- API Gateway
- Application Load Balancer (ALB)
- 独自のアプリケーションまたはコード

フェデレーティッドアイデンティティ (ID プールとも呼ばれる) は、ユーザーの一意の ID を作成し、アイデンティティプロバイダーとフェデレーションしますが、IdP としては機能しません。アイデンティティプロバイダーの例として次が挙げられます。

- Amazon Cognito ユーザープールやその他の OIDC プロバイダー
- パブリックプロバイダー (Facebook や Google など)
- SAML アイデンティティプロバイダー
- デベロッパーが認証したアイデンティティ

ID プールは、API Gateway や Application Load Balancer を介してリクエストをトンネルする場合よりも、Amazon Simple Storage Service (Amazon S3) や Amazon DynamoDB などの AWS サービスと直接統合する場合に便利です。

■ ユーザープールは、OIDC 規格に従い3 種類の JWT を発行します

アイデンティティ トークン

- ユーザーを認証する。ユーザーのアイデンティティ (名前や電子メールなど) に関するクレームが含まれる。



署名付き

アクセストークン

- 指定されたアクセス保護されたリソースのカスタムスコープに基づいて API コールを認可する。



署名付き

更新トークン

- 新しい ID またはアクセストークンの取得に必要な情報が含まれる。



暗号化

ユーザープールを使ってサインインすると、アイデンティティがどこにあるか (プール内またはフェデレーティッドサードパーティー IdP から送信される) に関係なく、Amazon Cognito から 3 つの JWT が発行されます。

- ID
- アクセス
- 更新

ID とアクセストークンは、署名付きで暗号化はされていません。リフレッシュトークンは暗号化されています。

デベロッパーはその役割として、ID とアクセストークンを取得してそれらを渡し、アプリケーションリソースへのアクセスを許可します。

バックグラウンドで ID とアクセストークンをサイレントに更新するには、クライアントはリフレッシュトークンを保存する必要があります。

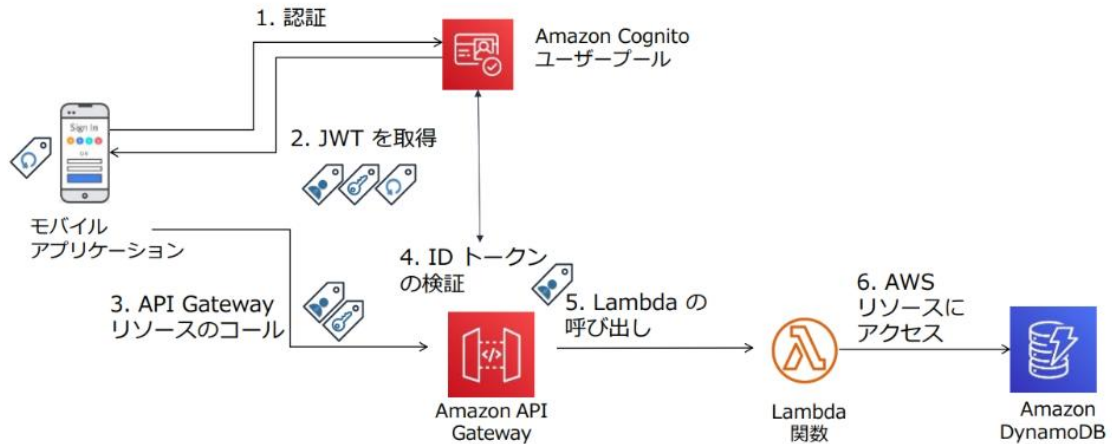
トークンの有効期間は比較的短期間に設定されています。アクセストークンとリフレッシュトークンの有効期間を選択できます。

- アクセストークンの有効期限は、短くて 5 分、長いものでは 24 時間に設定できます。
- リフレッシュトークンの有効期限は、短いもので 1 時間、長いものでは 10 年に設定できます。

これらのトークンは、アイデンティティに関するデベロッパーの作業をシンプル化し、開発者がこのユーザーコンテキストを容易に取得できるように、ダウンストリームのサービスに軽量なユーザーコンテキストを渡すことを可能にします。

ユーザープールディレクトリを IdP として使用する場合、AWS はエクスペリエンス全体をホストします。サードパーティー IdP とのフェデレーションを前提としたユーザープールを設定した場合、Amazon Cognito ユーザープールはこれらのアイデンティティを信頼し、あたかもユーザーがディレクトリに存在するかのようリダイレクトします。

例: Amazon Cognito ユーザープールを使用した REST API の認証フロー

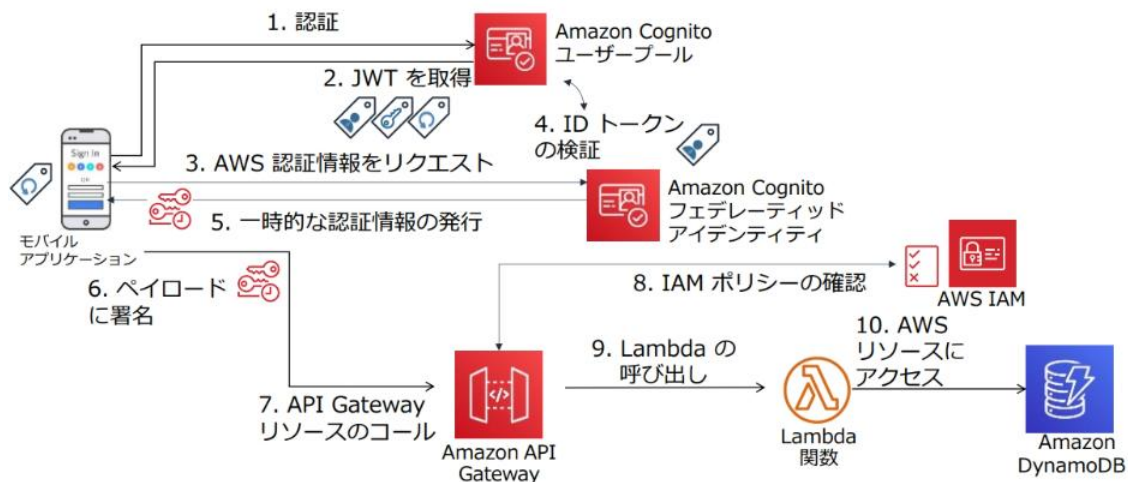


この図は、Amazon Cognito ユーザープールを使った認可フローを示しています。クライアントは最初にユーザープールを使用して認証し、それと引き換えに 3 つの JWT を取得します。次に、クライアントは、API Gateway への呼び出しの一部として、ヘッダーで ID とアクセストークンを渡します。API Gateway は、バックエンドで統合するリソース（この例では Lambda 関数）を呼び出す前に、トークンを検証します。API とアプリケーションの記述の仕方によっては、API Gateway が ID トークンまたはアクセストークンを渡す場合もあります。許可されたアクセスの範囲をさらに絞り込む必要がない場合は、ID トークンを渡します。

アクセスの範囲をさらに絞り込む必要がある場合は、アクセストークンを使用し、OAuth スコープを使って API メソッドリクエスト内で定義済みの属性を構成できます。

HTTP API のフローもこれと似ていますが、JWT トークンを発行するのが Amazon Cognito ユーザープールではなくサードパーティー JWT オートライザーである点異なります。

例: Amazon Cognito フェデレーティッドアイデンティティを使った IAM アクセス許可の付与



この例では、IAM ベースで認可を行うフェデレーティッドアイデンティティを使用して、一時的な IAM 認証情報を発行します。

クライアントは Amazon Cognito ユーザープールから JWT を取得し、それらを使って一時的な AWS 認証情報をリクエストします。Amazon Cognito フェデレーティッドアイデンティティは、ID トークンを検証した後、一時的な認証情報を返します。

クライアントは、これらの認証情報を使用してペイロードに署名し、API を呼び出します。

API Gateway は、一時的な認証情報に関連付けられた IAM ポリシーを確認し、ポリシーに規定されるアクションを許可します。

04 : サーバーレスデプロイの フレームワーク

Chapter 4

Infrastructure as Code で考慮すべき2つのプログラミングパラダイム

宣言型	命令型
望ましい状態を定義: 抽象的	それを行うための手順を定義: 手続き的
参入の障壁が低い より高度な言語やツールチェーンよりも必要な知識が少ない	参入の障壁が高い 作成者は、言語の構文、さまざまな API ライブラリ、および規則を理解している必要があり、例外を適切にキャッチできることが期待される
開発者の柔軟性が低い 完全なランタイムコントロール、ループ構造、またはオブジェクト指向の継承、スレッド化、自動テストといった高度な技術へのアクセスがない	開発者の柔軟性が高い 言語固有のエディタおよびツールは、コーダーの生産性を向上させるように設計されている

特定のサービスについて見ていく前に、コードを書くためのアプローチとして、宣言型と命令型の2つのアプローチを明確にしておきましょう。

■ 作業を簡素化して次のことを行う

- ◆ Infrastructure as Code の記述
- ◆ 多数のマイクロサービスやコンポーネントを個別にビルド、デプロイする
- ◆ 機能とリソースをパッケージ化し、サーバーに接続せずにクラウドにデプロイする

■ AWS での選択肢

- ◆ AWS CloudFormation
- ◆ AWS CDK
- ◆ AWS SAM
- ◆ Amplify フレームワーク

多数のマイクロサービスおよびコンポーネントを個別に構築できるようにするための新しいプラクティスの1つとして、フレームワークを使用してリソースとコードの展開を簡素化する方法です。デプロイ用にコンパイルされたバイナリを複数のサーバーに渡す必要はありません。代わりに、関数コードの取得先を AWS Lambda に伝える必要があります。また、アプリケーションに必要なその他のリソース (API、テーブルなど) をセットアップし、アプリケーションを構成するサービス間で適切なアクセス許可が設定されていることを確認する必要があります。サーバーに接続せずに、関数とリソースをパッケージ化してクラウドにデプロイする必要があります。

4 つの AWS での選択肢について説明しますが、市場には他にもフレームワークがあるため、既存のツールセットとうまく統合でき、必要な開発タイプに適したアプローチを選択するようにしてください。

CloudFormation で Infrastructure as Code を実現

- インフラストラクチャを宣言するための共通言語を提供
- YAML または JSON のテンプレートをサポート
- テンプレートから CloudFormation スタックを作成することにより、リソースのプロビジョニングと継続的な更新を自動化
- 宣言型と命令型の両方に対応

宣言オプション:

- 基本的な YAML/JSON
- Transforms
- Macros
- Include
- AWS サーバーレスアプリケーションモデル (AWS SAM)

命令オプション:

- AWS Cloud Development Kit (AWS CDK) (Python、TypeScript、Java、C#)
- Troposphere (Python)
- SparkleFormation (Ruby)
- GoFormation (GoLang)

CloudFormation により、テンプレートを使用してスタックを作成する



コード

- YAML または JSON でテンプレートを書く



コミット

- Amazon S3 バケットにテンプレートを保存する



作成

- コンソール、AWS CLI、または AWS SDK を使用してテンプレートからスタックを作成する
- スタックセットを使うと、複数のアカウント、複数のリージョンにデプロイ可能



デプロイ

- スタック、スタックセット、リソースを統一された方法で作成および管理する

CloudFormation: テンプレートは宣言型プログラミングを使用する

```
AWSTemplateFormatVersion: 2010-09-09
```

```
Parameters:
```

```
  DatabaseName:
```

```
    Type: String
```

```
Resources:
```

```
  MyDatabase:
```

```
    Type: AWS::RDS::DBInstance
```

```
    Properties:
```

```
      DBName: !Ref DatabaseName
```

```
      AllocatedStorage: '5'
```

```
      DBInstanceClass: db.m5.large
```

```
      Engine: MySQL
```

```
      DeletionPolicy: Snapshot
```

パラメータは、プロビジョニング時に指定できる入力変数

リソースは、インフラストラクチャスタックを構成するリソースと、対応する設定のリスト

YAML と JSON は、望ましい状態を宣言するために使用されます。

AWS CloudFormation に実行するための手順を明示的に伝えるのではなく、リソースの望ましい状態を何にするかを示します。

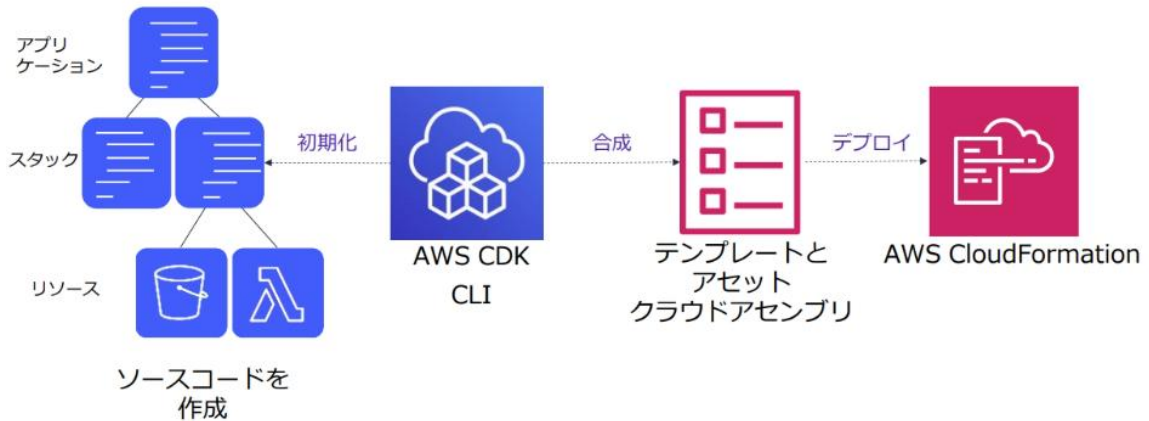
CloudFormation テンプレートは、宣言型プログラミングを使用します。

テンプレートでは、望ましい状態を宣言します。それを達成する手順は明示的に述べません。

■ AWS CDK: 命令型で Infrastructure as Code を実現

- プログラミング言語の親しみやすさと表現力を利用して、アプリケーションを命令的にモデル化する
- 多くの AWS サービスおよび機能に対応する構成ライブラリを提供する
- CloudFormation でリソースをプロビジョニングする

■ IDE で AWS CDK を使用して CloudFormation がデプロイするコードを記述する



AWS CDK アプリケーションは、AWS CDK を使用して AWS インフラストラクチャを定義する TypeScript、JavaScript、Python、Java、または C# で記述されたアプリケーションです。アプリケーションは、1 つ以上のスタックを定義します。

スタック (CloudFormation スタックに相当) には、コンストラクトが含まれています。

各コンストラクトは、Amazon Simple Storage Service (Amazon S3) のバケットや Lambda 関数など、1 つ以上の具体的な AWS リソースを定義します。

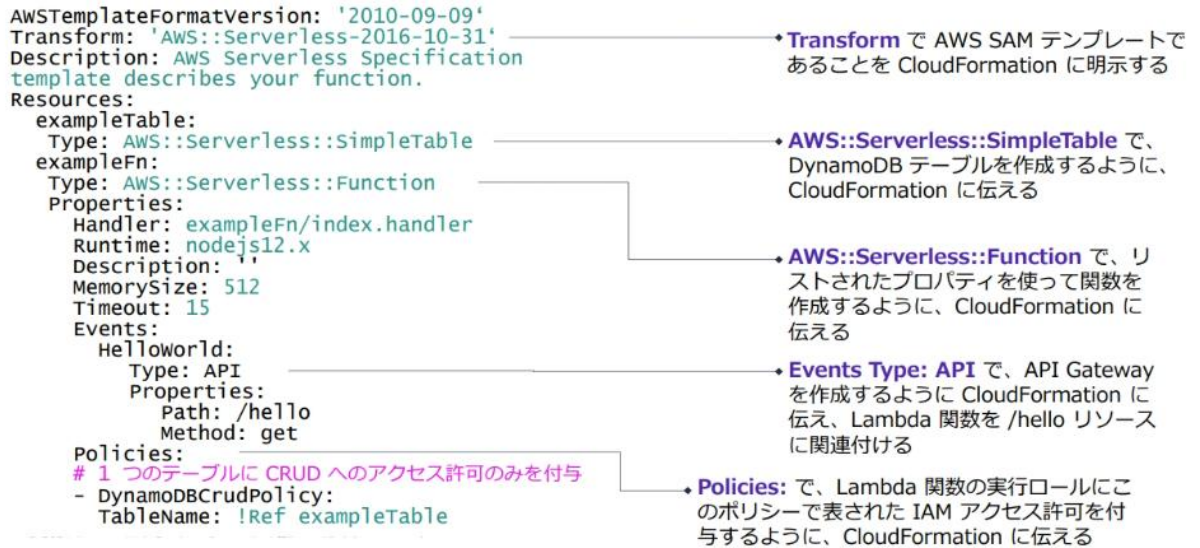
AWS CDK ツールキット (CLI コマンド `cdk` と呼ばれます) は、AWS CDK アプリケーションおよびスタックを操作するためのコマンドラインツールです。ツールキットには、1 つ以上の AWS CDK スタックを CloudFormation テンプレートおよび関連アセット (合成と呼ばれるプロセス) に変換し、スタックを AWS アカウントにデプロイする機能があります。

■ AWS SAM テンプレートでサーバーレスアプリケーションのInfrastructure as Code を簡素化

- サーバーレスリソースタイプを追加
 - ◆ Lambda 関数、レイヤー、アプリケーション
 - ◆ API (API Gateway)
 - ◆ テーブル (DynamoDB)
 - ◆ ステートマシン (Step Functions)
- CloudFormation でサポートされているすべてのリソースを含めることができる
- パラメータ、マッピング、出力、グローバル変数、組み込み関数、および一部のインポート値をサポート
- YAML または JSON で記述される

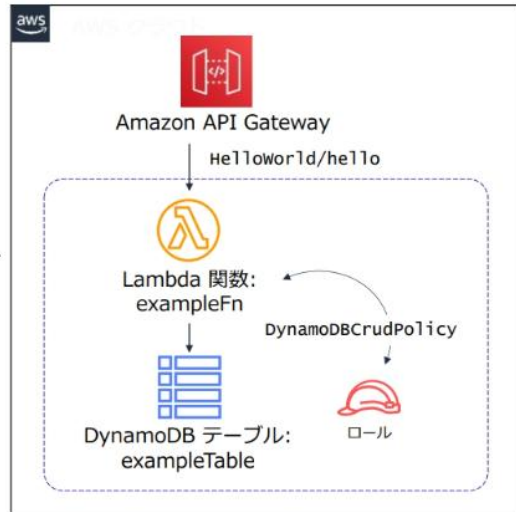
AWS SAM は、サーバーレスアプリケーションを構築するためのオープンソースのフレームワークです。迅速に記述可能な構文で関数、API、データベース、イベントソースマッピングを表現できます。

■ Transform で、これが変換する必要がある AWS SAM テンプレートであることを CloudFormation に伝える



CloudFormation は AWS SAM テンプレートを変換し、スタックを構築する

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: 'AWS::Serverless-2016-10-31'
Description: AWS Serverless Specification
             template describes your function.
Resources:
  exampleTable:
    Type: AWS::Serverless::SimpleTable
  exampleFn:
    Type: AWS::Serverless::Function
    Properties:
      Handler: exampleFn/index.handler
      Runtime: nodejs12.x
      Description: ''
      MemorySize: 512
      Timeout: 15
    Events:
      HelloWorld:
        Type: API
        Properties:
          Path: /hello
          Method: get
    Policies:
      # 1 つのテーブルに CRUD へのアクセス許可のみを付与
      - DynamoDBCrudPolicy:
          TableName: !Ref exampleTable
```

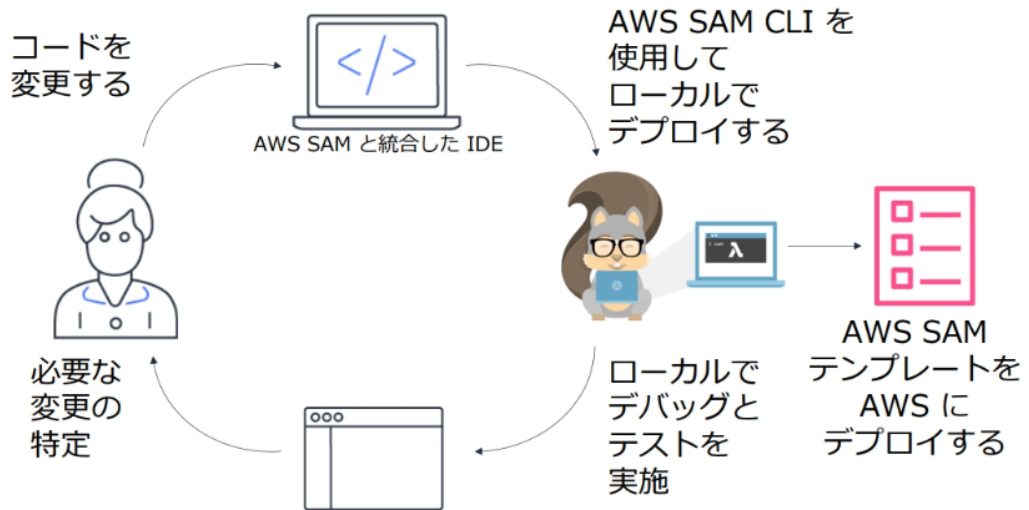


AWS SAM テンプレート構文ですべてのリソースを作成できるわけではありませんが、CloudFormation 構文を使用して他のリソースを作成または更新することができます。

■ AWS SAM CLI により、サーバーレスアプリケーションをローカルで簡単にコード化、デバッグ、テストする

- API Gateway プロキシスタイルおよび
- Lambda サービス API のテストをサポート
- ローカルマシンで応答オブジェクトと関数ログを使用できるようにする
- オープンソースの Docker-Lambda イメージを使用して、Lambda の呼び出し環境を再現する
- サンプルイベントを生成可能
- CloudWatch Logs から本番ログを TAIL できる
- ネイティブの依存関係を構築するのに役立つ

■ AWS SAM CLI で迅速に反復処理し、コードをパッケージ化してデプロイする



AWS SAM CLI コマンドを使用すると、サーバーレスアプリケーションを実行、パッケージ化、ビルド、デプロイできます。

■ Amplifyフレームワークには、フルスタック構築のための 4 つのコンポーネントがある

- Amplify ライブラリ: アプリコードの統合に使用できるユースケース中心のクライアントライブラリ
- Amplify UI コンポーネント: 一般的なフロントエンド開発フレームワーク用の UIライブラリ
- Amplify CLI: バックエンドサービスを設定するためのコマンドラインインターフェイス
- Amplify ホスティング: 継続的な統合とデプロイのための Git ベースのワークフローを提供するAWS のサービス

クライアントライブラリと UI コンポーネントにより、アプリのクライアント側のコーディングを簡素化できます。次に例を示します。

認証

- サインイン、サインアウト、パスワードを忘れた場合
再試行
認証情報の更新

Amplify Datastore

- オフラインシナリオとオンラインシナリオ
モデルの生成

Amplify CLI はカテゴリベースであり、デフォルトのオプションにベストプラクティスが組み込まれています。次に例を示します。

- `amplify add auth`: CLI により、サードパーティーのフェデレーティッドログインの有無にかかわらず、デフォルト設定を選択してリソースを構成できます。
- `amplify add api`: CLI は GraphQL または REST API の選択をガイドし、デフォルトのリソースセットを設定します
- `amplify mock api`: ローカルのモックとテストを行うことができます

Amplify ホスティングは、単一ワークフローのフロントエンドとバックエンドのデプロイを簡素化します。次に例を示します。

- 機能ブランチのデプロイ
- カスタムドメインとグローバルな可用性
- プルリクエストプレビュー
- エンドツーエンドテスト
- パスワードで保護されたブランチ
- リダイレクト

■ サーバーレスアプリ全体で複数のフレームワークを使用する可能性が高い

フレームワーク	利点
AWS CloudFormation	• 宣言的なスタイル、何でもモデル化 • 幅広く導入
AWS CDK	• 命令型プログラミング、使い慣れた言語またはコードを書くのと同じ言語を使用する • AWS SAM が考慮していないバックエンドにも有用
AWS SAM	• サーバーレス用に構築され、Lambda および API Gateway のリソースに非常に強い • 新しいサービスのサポートは継続的に追加される • ローカルデバッグまたは反復のための AWS SAM CLI • CloudFormation ユーザーにとって使い慣れた構文
Amplify	• AWS の知識があまりなくても、デフォルト設定を使用して迅速に構築およびデプロイできる • モバイルアプリ、単一ページの Web アプリ、フロントエンドクライアント • GraphQL と AWS AppSync に最適

各フレームワークには、組織とアプリケーションにとって考慮すべき特有の強みがあります。おそらく、アプリケーションのすべての側面に対して単一のフレームワークを使用することはないでしょう。特徴と機能も引き続き追加されます。

【演習 1 サーバーレスのウェブアプリを構築する】

タスク 1：継続的デプロイを使用した静的ウェブホスティング

AWS Amplifyは静的ウェブコンテンツ（例: HTML、CSS、JavaScript、イメージなどのファイル）を公開・配信するのに最適なサービスです。エンドユーザーは AWS Amplify コンソール によって公開されているパブリックウェブサイトの URL を使用してサイトにアクセスします。サイトを使用できるようにするため、ウェブサーバーを実行したり他のサービスを使用したりする必要はありません。

①AWSマネジメントコンソールで使用するリージョンを選択し、切り替えます。

②AWS CLIを使用するため、開発環境にインストールされていることを確認します。
インストールしていない場合は、ダウンロード、インストールします。

③ソースコードを管理するには、AWS CodeCommit を使用します。そのためGitリポジトリを作成します。
AWSマネジメントコンソールで「AWS CodeCommit」コンソールを開きます。

④[リポジトリを作成] を選択します。
[リポジトリ名] には「wildrydes-site」と入力します。
[作成] を選択します。

⑤リポジトリが作成されたら、IAM コンソールで Git の認証情報を使用し IAM ユーザーをセットアップします。
IAMページでユーザーを新規作成します。既存ポリシー（[AWSCodeCommitFullAccess]）をアタッチし、マネジメントコンソールへのログインも許可するため任意のパスワードを設定します。

⑥開発環境にGitがインストールされていない場合は、「https://git-scm.com/install/」からインストールします。

⑦IAM ユーザー用の Git 認証情報を作成します。作成済のIAMユーザーを選択し、「セキュリティ認証情報」タブの「AWS CodeCommit の HTTPS Git 認証情報」を作成、認証情報ファイルをダウンロードします。

⑧AWS CLI がインストールされたターミナルウィンドウ（コマンドプロンプト）で、「aws configure」を入力します。
作成済のIAMユーザーの AWS アクセスキー ID とシークレットアクセスキーを入力します。（Code Commitの認証情報ではありません）
デフォルトのリージョン名には、CodeCommit リポジトリを作成するために最初に選択したリージョンを入力します。
デフォルトの出力形式を空白のままにして、Enter キーを押します。

⑨git config 認証情報ヘルパーを設定します。以下のコマンドを実行します。
git config --global credential.helper "!aws codecommit credential-helper \$@"
git config --global credential.UseHttpPath true

※これにより.gitconfig ファイルが生成され、次の情報が書き込まれます。Git が AWS CodeCommit にアクセスする際、ユーザー名・パスワードの代わりに IAM 認証情報が自動的に使われます。

```
[credential]  
  helper = !aws codecommit credential-helper $@  
  UseHttpPath = true
```

⑩「git config --global --edit」（それぞれハイフン2個）コマンド を実行して、前の値がユーザープロファイルの .gitconfig file（デフォルトでは %HOME%¥.gitconfig または drive:¥Users¥UserName¥.gitconfig）に書き込まれていることを検証します。
成功した場合は（Git グローバルコンフィグレーションファイルにすでに存在する値に加えて）以前の値が表示されます。終了するには、通常、:q と入力して [Enter] を押すか、開いたエディタを閉じます。

⑪作成したIAMユーザーでAWSコンソールへログインし、使用リージョンに切替、AWS CodeCommit ページの wildrydes-site リポジトリを選択します。

⑫[URL のクローン] ドロップダウンから [HTTPS のクローン] を選択し、HTTPS URL をコピーします。

⑬ターミナルウィンドウから 「git clone」 コマンドを実行し、リポジトリの HTTPS URL を貼り付けます。
例) git clone <https://git-codecommit.us-east-1.amazonaws.com/v1/repos/wildrydes-site>

※「warning: You appear to have cloned an empty repository.」（空のリポジトリを複製した）という警告が表示されますが、想定通りです。

※認証情報が要求された場合は、作成済のIAMユーザーのGit認証情報
（ダウンロード済の「ユーザー名_codecommit_credentials.csv」ファイルの情報）を入力します。

⑭Gitリポジトリを設定します。

ローカルでクローンを作成したら、ハンズオンのために事前に用意されたS3 バケットからウェブサイトのコンテンツをコピーし、リポジトリにコンテンツを追加する必要があります。

そのため作成済IAMユーザーがS3へアクセスできる必要があるため、「AmazonS3FullAccess」ポリシーを追加設定します。

⑮以下コマンドを実行します。

```
cd wildrydes-site  
aws s3 cp s3://ttt-wildrydes/wildrydes-site ./ --recursive
```

⑯git ファイルを追加、コミット、プッシュします。

```
git add . （addの後ろにドットがあります）  
git commit -m "new files"  
git push
```

※「git commit」で「Author identity unknown」エラーが出る場合は、メッセージに従い「git config --global user.email」と「git config --global user.name」を実行します。

※「git push」でエラーが出る場合は、「git remote -v」コマンドで正しいURLかを確認。異なる場合は、「git remote set-url origin 自身のURL (<https://git-codecommit>・・・)」コマンドを設定して再実行してください。

⑰AWS Amplify コンソールを使用して、先ほど Git にコミットしたウェブサイトをデプロイします。Amplify コンソールが静的ウェブアプリケーションコードを保存するための場所のセットアップ作業を実施し、アプリケーションのライフサイクルを簡略化します。

AWS Amplify コンソールを起動し、[新しいアプリを作成] を選択します。

⑱[AWS CodeCommit] を選択し、次へ をクリックします。

⑲[リポジトリの選択] ドロップダウンから [wildrydes-site] を選択します。

⑳[ブランチ] ドロップダウンで [master] を選択し、[次へ] を選択します。

㉑[ビルド設定] ページでは、すべてデフォルトのままにして、[次へ] を選択し、作成します。

㉒Amplify コンソールで必要なリソースが作成され、コードをデプロイするため、このプロセスには数分かかります。

㉓「デプロイ済」になったことを確認し、指定されたURLをクリックしサイトを起動します。



②4 AWS Amplify コンソールは接続されたリポジトリで変更が検出されると、アプリを再ビルドし、再デプロイします。このプロセスを確かめるためにメインページに変更を加えてみます。

ローカルマシンで、wildrydes-site フォルダに移動し、任意のテキストエディタで index.html ファイルを開きます。

②5 タイトル行を次のテキストで変更します

```
<title>Wild Rydes - Rydes of the Future!</title>
```

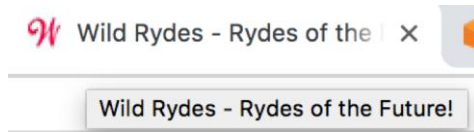
②6 ファイルを保存します。

②7 ターミナルウィンドウで、変更を追加してコミットし、変更を Git リポジトリに再度プッシュします。

Amplify コンソールはリポジトリへの変更を検出するとすぐにサイトのビルドを開始します。

「デプロイ済」から「デプロイ」にステータスが変更されています。

②8 Amplify が再デプロイを完了したら、Wild Rydes サイトを再度開いて、タブのタイトルが変更されていることを確認します。



タスク 2：ユーザーを管理する

ユーザーのアカウントを管理するための Amazon Cognito ユーザープールを作成します。顧客が新しいユーザーとして登録し、自分の E メールアドレスを確認して、そしてサイトにサインインできるようにするページを配置します。

ユーザーがウェブサイトアクセスすると、最初に新しいユーザーアカウントを登録します。登録のために E メールアドレスとパスワードを提供するようユーザーに要求します。ユーザーが登録を送信した後、Amazon Cognito は提供されたアドレスに確認コードを記載した確認メールを送信します。ユーザーのアカウントを確認するには、ユーザーはサイトに戻り、自分の E メールアドレスと受け取った確認コードを入力します。

ユーザーが確認済みアカウントを取得した後（コンソールでの E メール確認プロセスまたは手動確認のいずれかを使用）、サインインできるようになります。ユーザーがサインインするときに、ユーザー名（または E メール）とパスワードを入力します。その後、JavaScript 関数が Amazon Cognito と通信し、セキュアリモートパスワードプロトコル (SRP) を使用して認証して、一連の JSON ウェブトークン (JWT) を受け取ります。JWT にはユーザーの ID に関するクレームが含まれており、Amazon API Gateway で構築した RESTful API に対して認証するために次のステップで使用されます。

Amazon Cognito は、ユーザーを認証するための 2 つの異なるメカニズムを提供します。Cognito ユーザープールを使用してアプリケーションにサインアップおよびサインイン機能を追加するか、Cognito ID プールを使用して、Facebook、Twitter、Amazon などのソーシャル ID プロバイダー経由で、SAML ID ソリューションを使用し、ユーザーを認証することもできます。本演習では、提供された登録ページとサインインページのバックエンドとしてユーザープールを使用します。

① Amazon Cognito コンソールで、[ユーザープールを作成] を選択します。

② [アプリケーションのタイプ] では「シングルページアプリケーション (SPA)」を選択します。

③ アプリケーション名に「WildRydesWebApp」と入力し、「サインイン識別子のオプション」では「ユーザー名」を選択します。

④ 「サインアップのための必須属性」では「email」を選択し、ユーザーディレクトリを作成します。

⑤ ウェブサイトの設定ファイルを更新します。

前演習で入手済の「/js/config.js ファイル」には、ユーザープール ID、アプリケーション ID、およびリージョンの設定が含まれています。前のステップで作成したユーザープールとアプリの設定でこのファイルを更新し、ファイルをバケットにアップロードします。

ローカルマシンで任意のテキストエディタを使用し、wildryde-site/js/config.js ファイルを開きます。

⑥ ファイルの cognito セクションを、Cognito の [ユーザープール ID] と [アプリケーション ID] の正しい値に更新します。

region の値は、ユーザープールを作成した AWS リージョンのコードである必要があります。

例)

```
1 window._config = {
2   cognito: {
3     userPoolId: 'us-west-2_uXboG5pAb', // e.g. us-east-2_uXboG5pAb
4     userPoolClientId: '25ddkmj4v6hfsfvruhpfi7n4hv', // e.g. 25ddkmj4v6hfsfvruhpfi7n4hv
5     region: 'us-west-2' // e.g. us-east-2
6   },
7   api: {
8     invokeUrl: '' // e.g. https://rc7myt4tql.execute-api.us-west-2.amazonaws.com/prod',
9   }
10  };
```

⑦ 変更したファイルを保存します。

⑧ ターミナルウィンドウでファイルを追加、コミット、Git リポジトリにプッシュすると、自動的に Amplify コンソールにデプロイされます。

ブラウザでサイトにアクセスし、 [Giddy Up] ボタンをクリックします。

⑩登録フォームに入力して、[Let's Ryde] を選択します。自分の E メールを使うか、ダミーの E メールを入力することができます。少なくとも 1 つの大文字、数字、および特殊文字を含むパスワードを必ず選択してください。入力したパスワードを後で忘れることがないようにしてください。ユーザーが作成されたことを確認するアラートが表示されます。

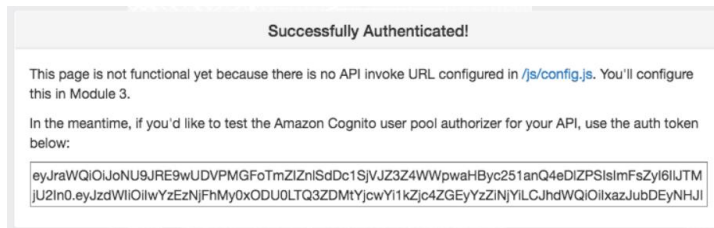
⑪次の2つの方法のいずれかを使用して、新しいユーザーを確認します。

a) 自分が管理する電子メールアドレスを使用した場合は、自分のウェブサイトドメインにある /verify.html にアクセスし、電子メールで受け取った認証コードを入力して、アカウントの検証プロセスを完了できます。確認メールは、スパムメールフォルダに分類される可能性があります。

b) ダミーの E メールアドレスを使用した場合は、Cognito コンソールから手動でユーザーを確認する必要があります。

- ・ Amazon Cognito コンソールで、[WildRydes] ユーザープールを選択します。
- ・ [ユーザー] タブで、登録ページを通して送信した E メールアドレスに対応するユーザーを確認する必要があります。ユーザーの名前を選択して、[ユーザー詳細] ページを開きます。
- ・ [アクション] ドロップダウンで [アカウントの確認] を選択し、アカウント作成プロセスを完了します。
- ・ [ユーザーアカウントの確認] ポップアップで、[確認] を選択します。

⑫/verify.html ページまたは Cognito コンソールを使用して新しいユーザーを確認した後、/signin.html にアクセスし、登録ステップで入力した E メールアドレスとパスワードを使用してログインします。成功した場合、/ride.html にリダイレクトされます。現在はまだ、「API が設定されていない」という通知が表示されます。



タスク 3：サーバーレスサービスバックエンド

AWS Lambda と Amazon DynamoDB を使用して、ウェブアプリケーションのリクエストを処理するためのバックエンドプロセスを構築します。ステップ 1 でデプロイしたブラウザアプリケーションを使用すると、ユーザーはユニコーンを自分の好きな場所に送信するように要求できます。これらのリクエストを満たすには、ブラウザで実行されている JavaScript がクラウドで実行されているサービスを呼び出す必要があります。

ユーザーがアプリでユニコーンを呼ぶと、そのたびに Lambda 関数が実行されます。この関数は、用意されているユニコーンの中から1つを選び、リクエスト内容を DynamoDB に保存します。その後、選ばれたユニコーンの情報をフロントエンドアプリに返します。この関数は、Amazon API Gateway を使用してブラウザから呼び出されます。

- ①Amazon DynamoDB コンソールで、[テーブルの作成] を選択します。
- ②[テーブル名] に「Rides」と入力します。このフィールドでは、大文字と小文字が区別されます。
- ③[パーティションキー] に「RideId」と入力し、キータイプとして [文字列] を選択します。このフィールドでは、大文字と小文字が区別されます。
- ④[テーブル設定] セクションで、[デフォルト設定] が選択されていることを確認し、[テーブルの作成] を選択します。
- ⑤[テーブル] ページで、テーブルの作成が完了するまで待ちます。完了すると、ステータスは [アクティブ] になります。
- ⑥テーブル名を選択します。
新しいテーブルの [概要] タブ > [一般情報] セクションで、[追加情報] を選択します。ARN をコピーします。これは次の手順で使用します。
- ⑦ログを Amazon CloudWatch Logs に書き込むアクセス許可と、項目を DynamoDB テーブルに書き込むアクセス許可を Lambda 関数に付与する IAM ロールを作成する必要があります。
IAM コンソールの左側のナビゲーションペインで [ロール] を選択し、[ロールの作成] を選択します。
- ⑧ [AWS サービス] を選択します。[ユースケース] で [Lambda] を選択し、[次へ] を選択します。
- ⑨フィルターテキストボックスに「AWSLambdaBasicExecutionRole」と入力し、Enter キーを押します。
AWSLambdaBasicExecutionRole ポリシー名の横にあるチェックボックスをオンにして [次へ] を選択します。
- ⑩[ロール名] に「WildRydesLambda」と入力します。他のパラメータはデフォルト設定のままにします。
[ロールの作成] を選択します。
- ⑪[ロール] ページのフィルターボックスに「WildRydesLambda」と入力し、先ほど作成したロールの名前を選択します。
- ⑫[アクセス許可] タブの [アクセス許可の追加] で [インラインポリシーを作成] を選択します。
[サービスの選択] セクションで、検索バーに「DynamoDB」と入力し、表示されたら [DynamoDB] を選択します。
[アクションの選択] を選択します。
[許可されているアクション] セクションで、検索バーに「PutItem」と入力し、表示されたら [PutItem] の横にあるチェックボックスをオンにします。
- ⑬[リソース] セクションで、[特定] オプションが選択されている状態で [ARN を追加] リンクを選択します。
[テキスト] タブを選択します。DynamoDB で作成したテーブルの ARN を貼り付け（前の手順で取得済）、[ARN を追加] を選択します。
- ⑭[次へ] をクリックします。
ポリシー名として「DynamoDBWriteAccess」と入力し、[ポリシーの作成] を選択します。

AWS Lambda は、HTTP リクエストなどのイベントに応答してコードを実行します。このステップでは、ウェブアプリケーションからの API リクエストを処理してユニコーンを特定するコア機能を構築します。次の手順では、Amazon API Gateway を使用して、ユーザーのブラウザから呼び出すことができる HTTP エンドポイントを公開する RESTful API を作成します。その後、このステップで作成した Lambda 関数をその API に接続して、ウェブアプリケーション用の完全に機能的なバックエンドを作成します。

AWS Lambda コンソールを使用して、API リクエストを処理する RequestUnicorn という名前の新しい Lambda 関数を作成します。

⑮ AWS Lambda コンソールで、[関数の作成] をクリックします。

⑯デフォルトの [一から作成] カードを選択したままにします。

⑰[関数名] フィールドに「RequestUnicorn」と入力します。

⑱[ランタイム] には [Python3.13] を選択します。

⑲[デフォルトの実行ロールの変更] ドロップダウンから [既存のロールを使用する] を選択します。

[既存のロール] ドロップダウンで [WildRydesLambda] を選択します。

[関数を作成] をクリックします。

⑳[コードソース] セクションまでスクロールし、「lambda_function.py」を以下のコードに置き換えデプロイします。

```
import json
import base64
import secrets
from datetime import datetime

import boto3

ddb = boto3.resource("dynamodb")
table = ddb.Table("Rides")

fleet = [
    {"Name": "Angel", "Color": "White", "Gender": "Female"},
    {"Name": "Gil", "Color": "White", "Gender": "Male"},
    {"Name": "Rocinante", "Color": "Yellow", "Gender": "Female"},
]

def lambda_handler(event, context):
    try:
        # 認証チェック (Cognito)
        authorizer = event.get("requestContext", {}).get("authorizer")
        if not authorizer:
            return error_response(
                "Authorization not configured",
                context.aws_request_id,
            )

        ride_id = to_url_string(secrets.token_bytes(16))
        print(f"Received event ({ride_id}): {event}")

        username = authorizer["claims"]["cognito:username"]

        body = json.loads(event["body"])
        pickup_location = body["PickupLocation"]
```

【参考：画面ショット】

```
lambda_function.py
1  import json
2  import base64
3  import secrets
4  from datetime import datetime
5
6  import boto3
7
8  ddb = boto3.resource("dynamodb")
9  table = ddb.Table("Rides")
10
11  fleet = [
12      {"Name": "Angel", "Color": "White", "Gender": "Female"},
13      {"Name": "Gil", "Color": "White", "Gender": "Male"},
14      {"Name": "Rocinante", "Color": "Yellow", "Gender": "Female"},
15  ]
16
17
18  def lambda_handler(event, context):
19      try:
20          # 認証チェック (Cognito)
21          authorizer = event.get("requestContext", {}).get("authorizer")
22          if not authorizer:
23              return error_response(
24                  "Authorization not configured",
25                  context.aws_request_id,
26              )
27
28          ride_id = to_url_string(secrets.token_bytes(16))
29          print(f"Received event ({ride_id}): {event}")
30
31          username = authorizer["claims"]["cognito:username"]
32
33          body = json.loads(event["body"])
34          pickup_location = body["PickupLocation"]
35
```

```

unicorn = find_unicorn(pickup_location)

record_ride(ride_id, username, unicorn)

return {
    "statusCode": 201,
    "body": json.dumps(
        {
            "RideId": ride_id,
            "Unicorn": unicorn,
            "Eta": "30 seconds",
            "Rider": username,
        }
    ),
    "headers": {
        "Access-Control-Allow-Origin": "*",
    },
}

except Exception as e:
    print(e)
    return error_response(str(e), context.aws_request_id)

def find_unicorn(pickup_location):
    print(
        "Finding unicorn for",
        pickup_location["Latitude"],
        ",",
        pickup_location["Longitude"],
    )
    import random

    return random.choice(fleet)

```

```

36 unicorn = find_unicorn(pickup_location)
37
38 record_ride(ride_id, username, unicorn)
39
40 return {
41     "statusCode": 201,
42     "body": json.dumps(
43         {
44             "RideId": ride_id,
45             "Unicorn": unicorn,
46             "Eta": "30 seconds",
47             "Rider": username,
48         }
49     ),
50     "headers": {
51         "Access-Control-Allow-Origin": "*",
52     },
53 }

54
55 except Exception as e:
56     print(e)
57     return error_response(str(e), context.aws_request_id)
58
59
60 def find_unicorn(pickup_location):
61     print(
62         "Finding unicorn for",
63         pickup_location["Latitude"],
64         ",",
65         pickup_location["Longitude"],
66     )
67     import random
68
69     return random.choice(fleet)
70

```



```
def record_ride(ride_id, username, unicorn):
    table.put_item(
        Item={
            "RideId": ride_id,
            "User": username,
            "Unicorn": unicorn,
            "RequestTime": datetime.utcnow().isoformat(),
        }
    )
```

```
def to_url_string(byte_data: bytes) -> str:
    return (
        base64.b64encode(byte_data)
        .decode("utf-8")
        .replace("+", "-")
        .replace("/", "_")
        .replace("=", "")
    )
```

```
def error_response(error_message, request_id):
    return {
        "statusCode": 500,
        "body": json.dumps(
            {
                "Error": error_message,
                "Reference": request_id,
            }
        ),
        "headers": {
            "Access-Control-Allow-Origin": "*",
        },
    }
```

```
71
72 def record_ride(ride_id, username, unicorn):
73     table.put_item(
74         Item={
75             "RideId": ride_id,
76             "User": username,
77             "Unicorn": unicorn,
78             "RequestTime": datetime.utcnow().isoformat(),
79         }
80     )
81
```

```
82
83 def to_url_string(byte_data: bytes) -> str:
84     return (
85         base64.b64encode(byte_data)
86         .decode("utf-8")
87         .replace("+", "-")
88         .replace("/", "_")
89         .replace("=", "")
90     )
91
```

```
92
93 def error_response(error_message, request_id):
94     return {
95         "statusCode": 500,
96         "body": json.dumps(
97             {
98                 "Error": error_message,
99                 "Reference": request_id,
100             }
101         ),
102         "headers": {
103             "Access-Control-Allow-Origin": "*",
104         },
105     }
106
```

② [テスト]タブ を選択し、[新しいイベントを作成] を選択し、[イベント名] フィールドに「TestRequestEvent」と入力します。

以下コードを[イベント JSON] セクションに入力します。

```
{
  "path": "/ride",
  "httpMethod": "POST",
  "headers": {
    "Accept": "*/*",
    "Authorization": "eyJraWQiOiJLTzRVMWZs",
    "content-type": "application/json; charset=UTF-8"
  },
  "queryStringParameters": null,
  "pathParameters": null,
  "requestContext": {
    "authorizer": {
      "claims": {
        "cognito:username": "the_username"
      }
    }
  },
  "body": "{\"PickupLocation\":{\"Latitude\":47.6174755835663,\"Longitude\":-122.28837066650185}}"
```

- ②テストが成功し、[詳細] ドロップダウンを展開、以下の結果となっていることを確認します。
(ユニコーンの種類はランダムのため、以下とは限りません)

```
{
  "statusCode": 201,
  "body":
    "{¥"RideId¥":¥"SvLnijlAtg6inAFUBRT+Fg==¥",¥"Unicorn¥":{"¥"Name¥":¥"Rocinante¥",¥"Color¥":¥"Yellow¥",¥"Gender¥":¥"Female¥"},
    ¥"Eta¥":¥"30 seconds¥"}",
    "headers": {
      "Access-Control-Allow-Origin": "*"
    }
}
```

タスク 4：RESTful API をデプロイする

Amazon API Gateway を使用して、前のモジュールで RESTful API として構築した Lambda 関数を公開します。この API はパブリックインターネットでアクセス可能になります。

最初のステップでデプロイした静的ウェブサイトには、このモジュールで構築する API と連携するように設定済みのページが既にあります。/ride.html のページには、ユニコーンに乗ることを要求するシンプルなマップベースのインターフェイスがあります。/signin.html ページを使用した認証後、ユーザーはマップ上のポイントをクリックし、右上隅の [Request Unicorn] ボタンを選択して、乗る場所を選択することができます。

- ①Amazon API Gateway コンソールで、[APIの作成] を選択します。
- ②[REST API] で [構築] を選択します。
- ③ [新しい API] を選択、[API 名] に「WildRydes」と入力します。
- ④[エンドポイントタイプ] ドロップダウンで [エッジ最適化] を選択します。
- ⑤[Security policy] では「TLS_1_0」を選択し、[API の作成] を選択します。
- ⑥Amazon Cognito ユーザープールオーソライザーを作成する必要があります。Amazon API Gateway は、Amazon Cognito ユーザープール から返される JSON ウェブトークン (JWT) を使用して API 呼び出しを認証します。ユーザープールを利用できるように、API のオーソライザーを作成します。

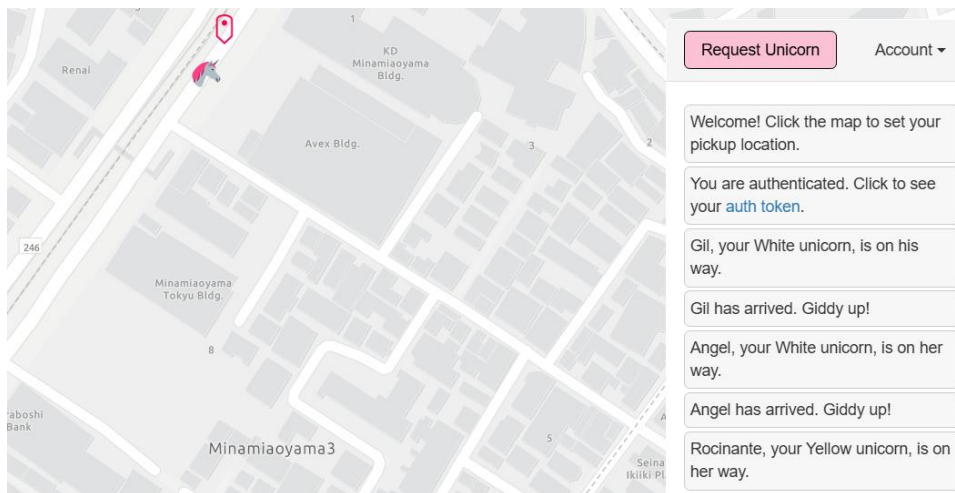
先ほど作成した WildRydes API の左側のナビゲーションペインで、「オーソライザー」を選択します。

- ⑦[新しいオーソライザーの作成] を選択します。
- ⑧オーソライザーの [名前] フィールドに「WildRydes」と入力します。
- ⑨[タイプ] として [Cognito] を選択します。
Cognito ユーザープールの [リージョン] ドロップダウンで、使用中のリージョンを選択します。Cognito ユーザープールの [名前] フィールドから「WildRydes」を選択します。
- ⑩[トークンのソース] に「Authorization」と入力し、[作成] を選択します。

API 内に新しいリソースを作成します。その後、そのリソース用の POST メソッドを作成し、このモジュールの最初のステップで作成した RequestUnicorn 関数によってサポートされる Lambda プロキシ統合を使用するようにメソッドを設定します。

- ⑪WildRydes API の左側のナビゲーションペインで、[リソース] を選択します。
- ⑫[アクション] ドロップダウンから、[リソースの作成] を選択します。
- ⑬[リソース名] に「ride」と入力すると、リソースパス /ride が自動的に作成されます。
- ⑭[API Gateway CORS を有効にする] のチェックボックスをオンにし、[リソースの作成] を選択します。
- ⑮新しく作成した /ride リソースが選択された状態で、[アクション] ドロップダウンから [メソッドの作成] を選択します。
- ⑯[オプション] の下に表示される新しいドロップダウンから [POST] を選択し、チェックマークアイコンを選択します。
統合タイプとして [Lambda 関数] を選択します。
[Lambda プロキシ統合の使用] のチェックボックスをオンにします。
- ⑰使用中のリージョンを選択し、[Lambda 関数] に「RequestUnicorn」と入力します。[保存] を選択します。

- ⑮[メッセージカード] カードを選択します。「編集」ボタンを押下し、
[認可] ドロップダウンの一覧から WildRydes Cognito ユーザープールオーソライザーを選択し、保存します。
- ⑯[アクション] ドロップダウンリストで [API のデプロイ] を選択します。
- ⑰[デプロイステージ] ドロップダウンリストから [新しいステージ] を選択します。
[ステージ名] に「prod」と入力します。
[デプロイ] を選択します。
- ⑱[呼び出し URL] をコピーします。これらは次ステップで使用します。
- ⑲ ウェブサイトのデプロイで /js/config.js ファイルを更新し、作成したステージの呼び出し URL を含めます。
ローカルマシンで js フォルダに移動し、任意のテキストエディタで config.js ファイルを開きます。
- ⑳ 前手順で Amazon API Gateway コンソールからコピーした呼び出し URL を config.js ファイルの invokeUrl 値に貼り付けます。 ファイルを保存します。
- ㉑ 更新した config.js ファイルを追加、コミット、Git リポジトリにプッシュすると、自動的に Amplify コンソールにデプロイされます。
- ㉒ 動作確認のため、ride.html ファイルの ArcGIS JS バージョンを 4.3 から 4.6 に次のように更新します。
(以下、2か所の変更部分は離れた箇所にあります)
- ```
<script src="https://js.arcgis.com/4.6/"></script>
<link rel="stylesheet" href="https://js.arcgis.com/4.6/esri/css/main.css">
```
- ㉓ 変更したファイルを保存します。 ファイルを追加、コミット、Git リポジトリに git プッシュすると、AWS Amplify コンソールに自動的にデプロイされます。
- ㉔ デプロイ完了後、ウェブサイトの /ride.html にアクセスします。  
マップがロードされたら、マップ上の任意の場所をクリックして、ピックアップ場所を設定します。
- ㉕ マップ右上の[Request Unicorn] を選択します。右のサイドバーの通知で、ユニコーンが向かっていることが表示され、続いてピックアップ場所にユニコーンのアイコンが移動します。

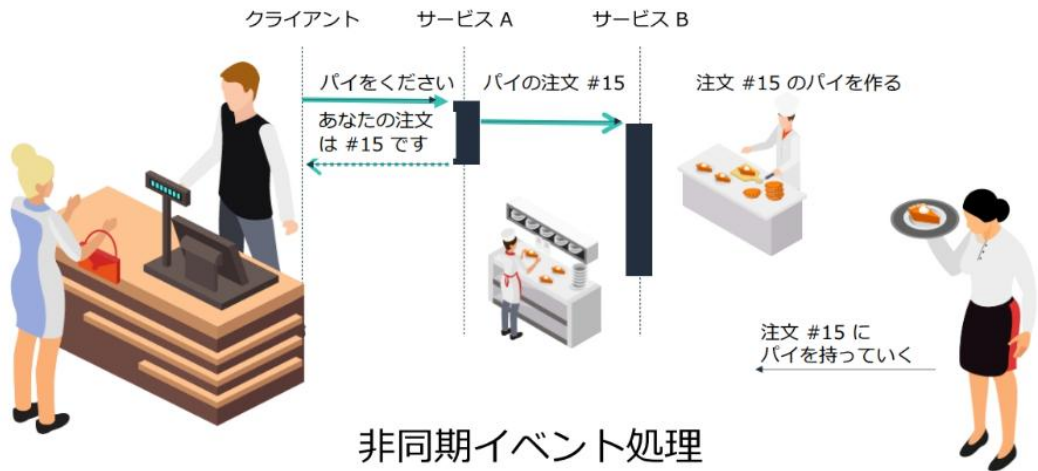


DynamoDBのテーブルにも項目が追加されていることを確認します。

# 05 : Amazon EventBridge と Amazon SNS を使用してコンポーネント を疎結合化する

## Chapter 5

## ■ 非同期イベントは、ダウンストリーム作業が完了するまで待たない



同期モデルとは対照的に、非同期の設計ではクライアントは要求を送信し、イベントが受信されたという確認を受け取ることがありますが、クライアントは要求の結果を含む応答を受信しません。クライアントがサービス A に要求を行うと、サービス A はコンシューマー（この例では、サービス B）にイベントを通知し、次に進みます。欠点は、サービス B がサービス A に情報を返すチャンネルがないことです。コードを記述するのに慣れている場合でも、明示的な結合はほとんど必要ありません。このアプローチの利点は、ダウンストリームアクティビティへの依存を減らすことで、クライアントへの応答性が向上することです。つまり、長い待ち時間を処理したり、ダウンストリームで発生する可能性のあるエラーを処理するために、ロジックをコードに組み込む必要はありません。クライアントが要求を正常に引き渡したら、先に進むことができます。非同期接続の観点で設計を行えば、はるかに柔軟で耐障害性のあるアプリケーションを作成できます。

ダイナーの例では、非同期イベントはパイのスライスを注文する顧客であり、ダウンストリームサービスではそれを用意する必要があります。カウンターアテンダントは、注文を確認して注文番号を生成し、ダウンストリームサービスに指示して、その注文番号に関連付けられたパイを用意します。パイの用意ができたなら、ダウンストリーム方式でパイをデリバリーするか、注文が準備できたことを顧客に知らせますが、これは同じチャンネルを介して戻ってくるわけではありません。

## ■ 非同期呼び出しでは、Lambda は関数の受信イベントをキューに入れる



Lambda には、非同期イベントのサポートが組み込まれています。

非同期呼び出しでは、Lambda はイベントを独自の内部イベントキューに置き、成功応答をクライアントに返します。Lambda サービス内の別の内部プロセスがキューからイベントを読み取り、関数に送信します。

Lambda がイベントをキューに追加することができない場合、エラーメッセージがコマンド出力に表示されます。

Lambda は内部イベントキューを管理し、関数がエラーを返した場合は、増加する間隔でイベントを自動的に 2 回再試行します。再試行をすべて使い果たした後、失敗した試行をデッドレターキューに送るか、失敗時の送信先に送信できます。詳細については、モジュール 7 と 8 を参照してください。

同時実行性がないために関数を呼び出せない場合、イベントが Lambda 内部キューで数時間または数日間待機している可能性があります。

開発者は Lambda キューを制御できませんが、失敗したアイテムを破棄するか別の宛先に送信する前に Lambda が再試行を実行する最大時間と、再試行回数 (0~2) を変更できます。

関数がエラーを返さない場合でも、キュー自体は最終的に一貫しているため、同じイベントを Lambda から複数回受信する可能性があります。関数が受信するイベントに対応できない場合、イベントは関数に送信されずにキューから削除される場合があります。

関数コードが重複イベントを適切に処理し、すべての呼び出しを処理するのに十分な同時実行性があることを確認してください。

AWS コマンドラインインターフェイス (AWS CLI) を使用して非同期で関数を呼び出すには、呼び出しタイプを「Event」に設定して `invoke` コマンドを使用します。

このコースでは、非同期イベントソースとして Amazon EventBridge と Amazon Simple Notification Service (Amazon SNS) について確認します。

EventBridge と Amazon SNS はどちらも Lambda の非同期イベントソースであり、1 つのイベントから複数の並列アクションを開始するのに適しています。どちらも、サブスクライバーのフィルタリングルールを作成できます。

## クライアントにステータスを知らせる

私の注文は用意できていますか？



クライアントの  
ポーリング

ご注文は用意できています。



Webhook

こちらがご注文の品です。



WebSocket

非同期通信では、ダウンストリームタスクがいつ実行されるかをクライアントが把握し、次のステップを完了できるようにすることが重要です。ダイナーの例では、顧客やウェーターは注文の準備ができていることをどのように知るのでしょうか。

通常、次の3つの方法があります。

- 顧客は、数分おきにパン屋のカウンターで注文を確認します。
- パン屋は、注文の準備ができたときに振動するデバイスを顧客に提供し、顧客は注文したものを受け取ります。
- 顧客はテーブルの上に置く番号を受け取り、準備ができたなら、ウェーターは顧客が注文したものを届けます。

これらは、クライアントのステータスを取得するための、およそ3つのオプションに相当します。

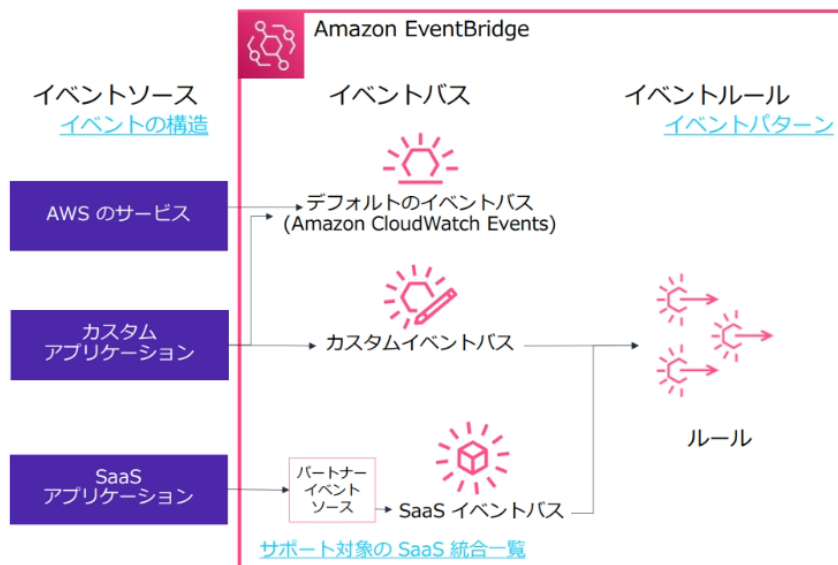
- クライアントは、イベントのステータスをポーリングします。
- クライアントは Webhook を使用して、イベントが完了したときに通知を受け取ります。
- クライアントは WebSocket を使用して双方向接続を行い、プロセスの結果をクライアントに直接配信できるようにします。



## ■ EventBridge により、イベント駆動型アプリケーションを簡単に構築できる

開発者向け機能	セキュリティ機能と拡張機能
• 独自のアプリケーション、AWS のサービス、または SaaS アプリケーションを含むソース    • イベントをフィルタするためのルーティングルール    • イベントのスケジューリング    • 多数のターゲット    • イベント構造を定義するためのスキーマの登録と検出	• スケーラブルなフルマネージド型    • AWS リソースに関するシステムイベントのストリームをほぼリアルタイムに配信    • 統合された CloudWatch メトリクス    • 統合された IAM サポート (リソースポリシーを含む)    • AWS の内部ネットワークを経由する SaaS 統合アプリケーションイベント

## ■ 仕組み



EventBridge を使用して、以下からイベントをルーティングできます。

- AWS のサービス
- アプリケーション内で作成するカスタムイベント
- EventBridge 統合を追加した SaaS アプリケーション

イベントソースは、設定したバスにイベントを配置します。

すべてのアカウントに、デフォルトでイベントバスが作成されます。これは、Amazon CloudWatch Events が使用するものであり、AWS サービスのイベントがルーティングされます。カスタムアプリケーションのイベントをデフォルトのバスに配置するか、カスタムイベントバスを作成することができます。

SaaS アプリケーションは、SaaS イベントバスに接続されたパートナーイベントソースを使用して、統合された SaaS アプリケーションからのトラフィックをルーティングします。それぞれ独自のカスタムイベントバスを使用します。

EventBridge は、サポートされている SaaS アプリケーションのデータを取り込み、まるで AWS のサービスが開始しているかのように、AWS のサービスターゲットにルーティングできます。これにより、AWS Identity and Access Management

(IAM) のクロスアカウントロールと認証情報を必要とせずに、SaaS と AWS アカウント間に論理的な接続が作成されます。

トラフィックは、パブリックインターネットではなく AWS 内部ネットワーク上を移動します。

EventBridge のイベントは JSON オブジェクトとして表されます。すべてのイベントは同じような構造をしており、最上位のフィールドも同じです。

イベントバスに配置されたイベントをフィルタリングしてルーティングするためのルールを作成します。ルールは JSON で表され、イベントパターンを使用してイベントバス上の AWS イベントを照合します。パターンが一致すると、ルールはそのイベントをターゲットにルーティングします。イベントパターンは、一致するイベントと同じ構造をしています。

## ■ イベント例・ルール例

### イベントの例

```
{
 "detail-type": "Order Created",
 "source": "aws.partner/example.com/orders",
 "detail": {
 "orderId": "987654321",
 "department": "billing",
 "creator": "user12345"
 ...
 }
}
```

### ルールの例

```
{
 "source": ["aws.partner/example.com/orders"]
}
```

### ルールの例

```
{
 "detail": {
 "department": ["billing", "fulfillment"]
 }
}
```

ここでは簡単な例をいくつか挙げます。

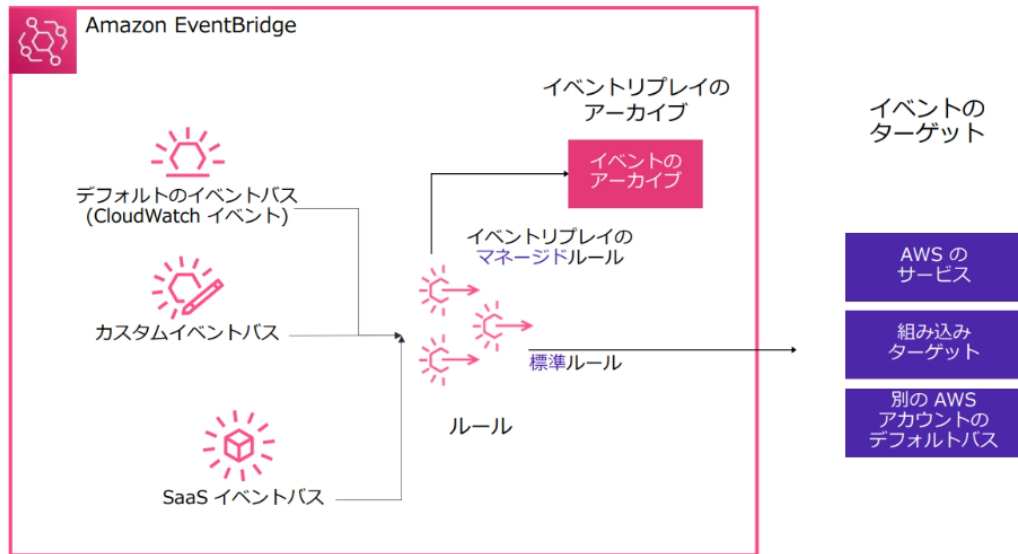
イベントには、常に詳細タイプ、ソース、イベントの詳細が含まれます。

これらの各コンポーネントに基づいてフィルターを作成できます。

この例では、1 つ目のルールはソースフィールドの一致を検索し、2 つ目のルールはイベントの詳細セクション内の特定の要素を検索します。

1 つ目は、ソースと一致するため、一致です。2 つ目は、ルールの詳細セクションにある部門の 1 つと一致するため、一致です。

## ターゲット



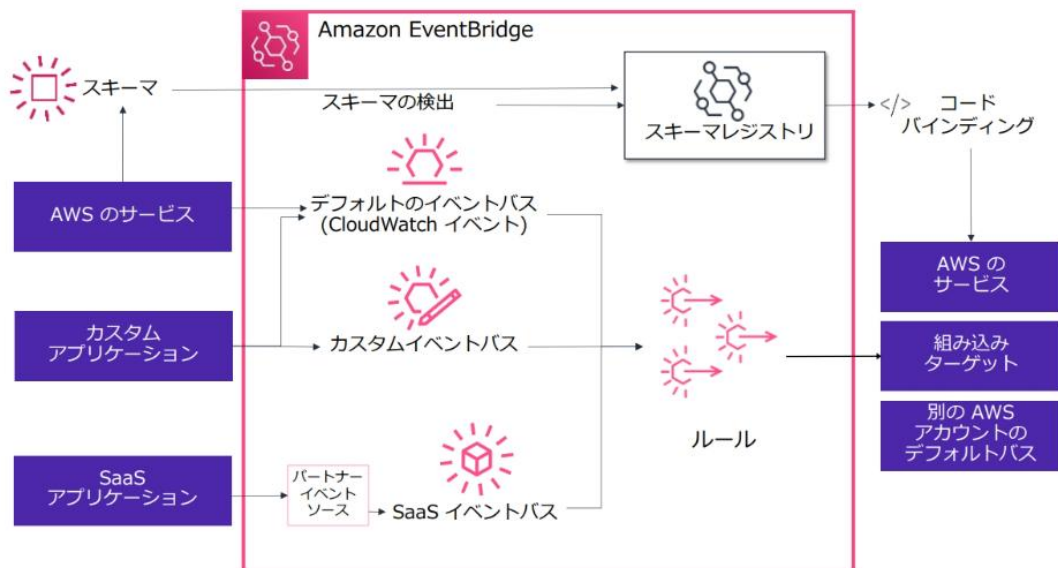
各ルールに複数のターゲットを関連付けることができます。

ダイナーの例に戻ると、ドーナツサービス、領収書サービス、在庫サービスはすべてドーナツの新しい注文に関連しているため、ドーナツのルールは各ターゲットに関連付けられます。

イベント駆動型アーキテクチャでは、サービスが過去のイベントにアクセスする際に役立ちます。例えば、システムエラーから回復する必要がある場合や、新しい機能を検証する場合などです。

このメカニズムの作成は、複雑になる可能性があります。EventBridge のアーカイブおよび再生機能を使用すると、任意のタイプのイベントバスが生成するすべてのイベントを記録することができます。再生機能では、記録されたイベントをアーカイブに保存します。ルールで使用したのと同じイベントパターンマッチングロジックを使用して、すべてのイベントを記録するか、アーカイブするイベントをフィルタリングするかを選択できます。

## ■ スキーマ



多数のソースに対してオプションを使用する場合、イベントをロードするときに何が期待されるのかを知るのが困難になります。

スキーマレジストリは、検索可能なレジストリにイベントスキーマを格納し、コード内で強く型指定されたオブジェクトとしてイベントを表現するために使用できるコードバインディングを生成します。これにより、コード内でイベントをオブジェクトとして簡単に使用することができます。

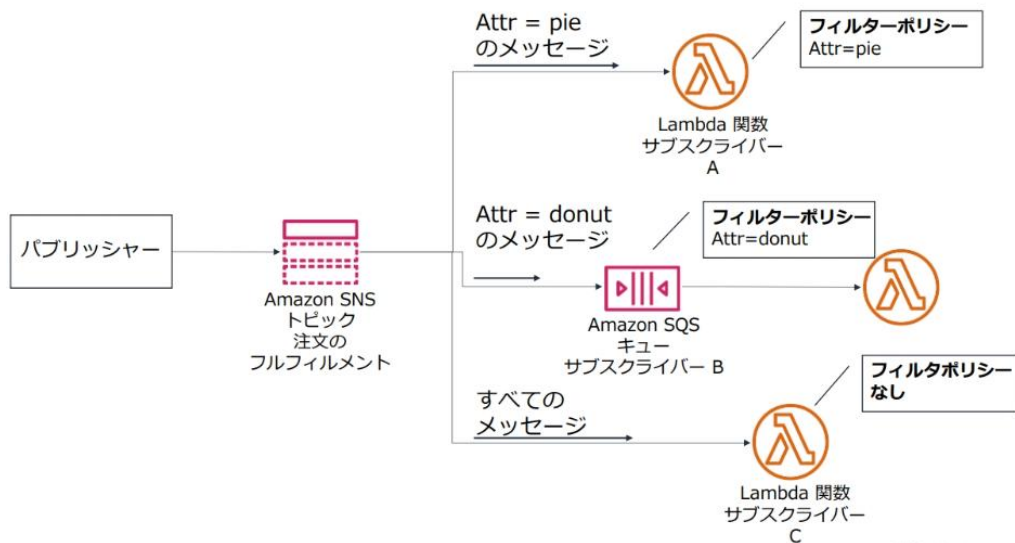
EventBridge のスキーマレジストリには、組織のアプリケーション、AWS のサービス、SaaS アプリケーションで作成したスキーマが含まれます。

スキーマレジストリを使用すると、すべての AWS のサービススキーマを可視化できます。また、スキーマの検出により、スキーマが自動的に検出され、レジストリに追加されます。

## ■ Amazon SNS により、コンポーネントをPub/Sub モデルで疎結合化する

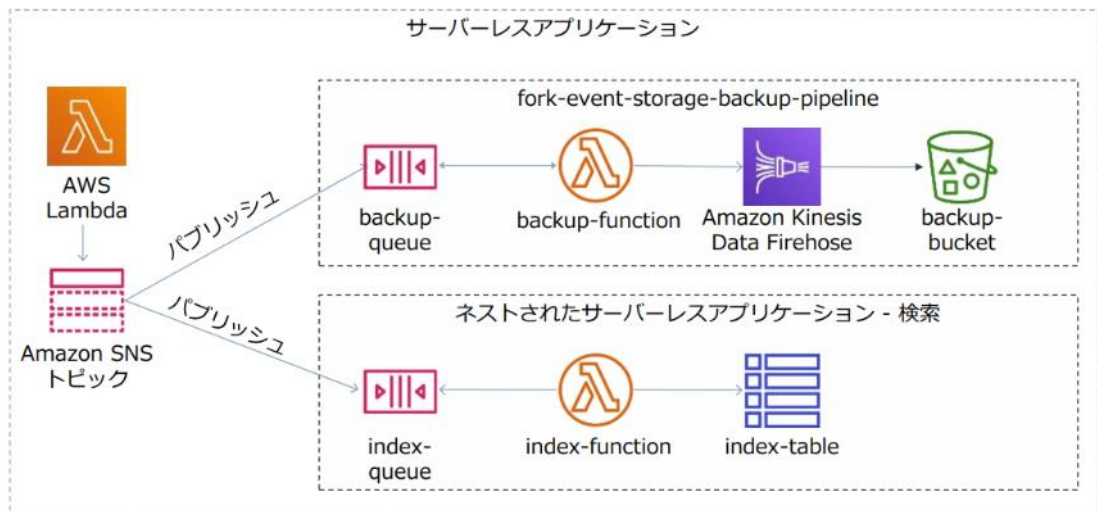
開発者向け機能	セキュリティ機能と拡張機能
• 多数のネイティブサービスの統合    • HTTP エンドポイントを呼び出す機能 (ポーリング、Webhook、WebSocket 用)    • サブスクライバ固有のフィルタリング    • 広範なファンアウトパターン    • サブスクライバとしてのイベント処理パイプライン    • 電子メールまたは SMS のサブスクライバ    • 命令されたメッセージ配信のための FIFO トピック	• 4 フェーズの再試行ポリシーとデッドレターキューを含む    • 受信したすべてのメッセージを永続的に保存    • 保管中のデータのサーバー側の暗号化をサポート    • Amazon VPC と Amazon SNS の間にプライベート接続を確立できる    • ログ配信のステータス

## ■ フィルタリングの仕組み



Amazon SNS でメッセージをフィルタリングし、サブスクライバーがフィルターポリシーをトピックのサブスクリプションに割り当てます。パブリッシャーには、トピックのメッセージの完全なコンテキストを生成するロジックが含まれ、属性をメッセージのペイロードとともにキーと値のペアとして提供します。フィルターポリシーにはサブスクライバーが受信するメッセージを定義する属性が含まれています。Amazon SNS はメッセージ属性を各サブスクリプションのフィルターポリシーと比較します。Amazon SNS は、サブスクリプションごとにメッセージ属性をフィルターポリシーと比較し、サブスクライバーにとって必要なメッセージのみを配信します。つまり、パブリッシャーはメッセージをサブスクライバーにルーティングする必要はなく、サブスクライバーは無関係なメッセージをフィルタリングする必要もありません。Amazon SNS により、各サブスクライバーは気になるメッセージを確実に受け取ることができます。

## Amazon SNS を使用してネストされたアプリケーションを作成する



Amazon SNS は、大規模なアプリケーション内でネストされたアプリケーションにファンアウトするのにも適しています。

アプリケーションの構築にパターンが出現すると、再利用可能な小さなアプリケーションを構築し、それらをネストされたアプリケーションとして大規模なアプリケーションに組み込むことができます。

この例で示される最上位アプリケーションは、AWS Serverless Application Repository を通じて利用可能な AWS Event Fork Pipelines の 1 つです。



## EventBridge と Amazon SNS のどちらを選択するか

SaaS の統合と複雑なルーティング ルール向けの EventBridge	非常に広範なファンアウトや FIFO 処理 向けの Amazon SNS
• SaaS アプリケーションとの統合    • ソースおよびターゲットとうまく統合でき、多くの選択肢がある    • より高度なルーティングルールだが、ビルドの作業には少し手間がかかる場合がある    • スキーマレジストリで、チームのイベントの書き込み方法を標準化する    • 保存期間は 24 時間	• 電子メールおよび SMS のサブスクライバー    • ポーリング、Webhook、WebSocket パターンのエンドポイントを呼び出す    • メッセージの順序を保持する FIFO トピックのオプション    • 非常に広範なファンアウト    • Lambda 関数のデッドレターキューとして機能できる    • 再試行ポリシーは数日にわたって延長される

EventBridge と Amazon SNS はどちらもアップストリームプロデューサーに非同期バッファを提供し、並列処理のために複数のコンシューマーにファンアウトできます。機能が更新されると、どのサービスが最適か明確に選択できない場合がありますが、いつでもケースバイケースで決定することができます。EventBridge には、多くのイベントソースやターゲットに対応するネイティブ統合があり、SaaS との統合も増加しています。Amazon SNS はそうではありません。

EventBridge で作成できるルールはより高度なものですが、Amazon SNS に比べて、設定の作業は少し手間がかかるかもしれません。

Amazon SNS は、SMS サブスクリプションや、ポーリングを使用するワークフロー (Webhook や WebSocket パターンなど) での使用に適しています。

Amazon SNS FIFO トピックは、メッセージの順序を維持する必要がある場合に適しています。例えば、Amazon SNS FIFO トピックを Amazon Simple Queue

Service (Amazon SQS) FIFO キューと一緒に使用して、できるだけ低コストでインベントリ管理などのタスクを実行することができます。

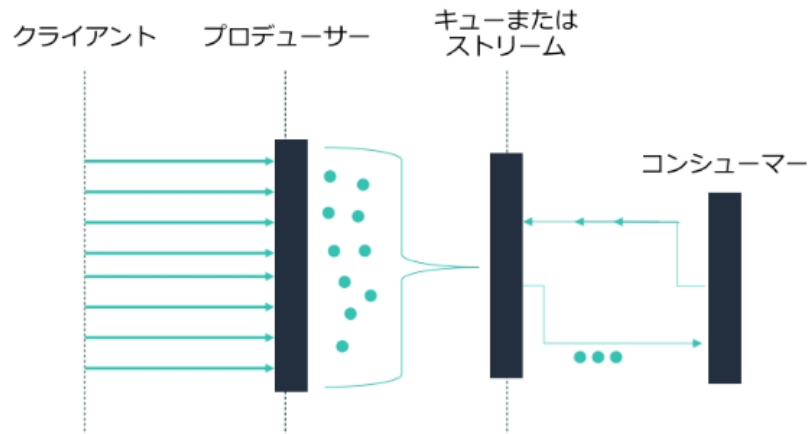
Amazon SNS では、EventBridge よりも多くのコンシューマーと低レイテンシーが可能です。どちらも非同期統合のスケーリング用に設計されています。

Amazon SNS では、HTTP エンドポイントに設定できる再試行ポリシーを使用して、再試行期間が延長されています。これは、停止しやすいエンドポイントや、信頼性の低いインフラストラクチャに存在するエンドポイントに接続する場合に特に有用です。

## 06：キューとストリームを使用したイベント駆動型開発

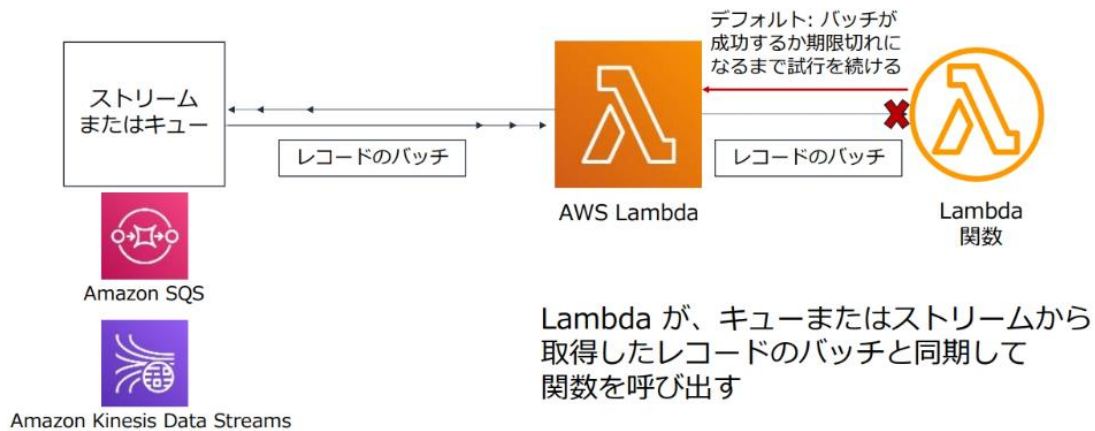
### *Chapter 6*

■ プロデューサーがイベントをキューまたはストリームに配置し、コンシューマーがレコードをポーリングする



ポーリングイベントでは、コンシューマーはバッチ内のメッセージをプロデューサーにポーリングし、新しいレコードのバッチを返す前にバッチを処理します。  
コンシューマーはバッチ内のメッセージをプロデューサーにポーリングし、新しいレコードのバッチを返す前にバッチを処理します。

## ■ Lambda がコンシューマーの場合、ユーザーに代わってキューまたはストリームをポーリングする



Lambda の前にキューまたはストリームを設定すると、Lambda がポーリングの作業を処理します。Lambda は、キューまたはストリームから取得したレコードのバッチと同期して関数を呼び出します。キューとストリームの両方で、Lambda にはユーザーが設定できる再試行動作が組み込まれていますが、Lambda はアプリケーションでキューまたはストリームコンシューマーを設定するために行う必要がある統合作業のほとんどを処理しています。

デフォルトでは、Lambda はバッチが成功するか、レコードの有効期限が切れるまで、レコードのバッチを再試行し続けます。

キューとストリームの Lambda 統合には、使用すべきエラー処理の設定が組み込まれています。

## ■ キューとストリーム

	キュー	ストリーム
データ値	値は、個々のメッセージの処理から得られる	値は、実行可能なデータを取得するためにメッセージを集約することから得られる
メッセージレート	メッセージレートは可変	メッセージレートは連続的で大量
メッセージの処理	メッセージは単一のコンシューマーを対象としており、正常に処理された後に削除される	メッセージは複数のコンシューマーが並列処理で使用でき、各コンシューマーはポインタを保持する必要があるが、レコードは削除されない
デフォルトの再試行動作	失敗したメッセージは、キューに再び表示され、将来のバッチで再試行される。メッセージの次のバッチ処理が続行される	失敗したメッセージにより処理がブロックされる。失敗したバッチは、メッセージが成功するか期限切れになるまで再試行される

キューとストリームは、さまざまなタイプのデータパターンに適しており、それぞれ少し異なる方法でデータを処理します。

キューを使用した処理

- 個々のメッセージは、処理またはコンピューティングを適用するコアエンティティです。これは、金融取引かもしれませんし、あるいはモノのインターネット (IoT) デバイスを制御するためのコマンドかもしれません。

- 通常、メッセージの発生はさまざまです。

メッセージは、処理後にキューから削除する必要があります。

キューの再処理ポリシーは、レコードが再試行を取得する回数を制御します。また、設定した最大受信回数を超えるレコードについては、デッドレターキューを Amazon Simple Queue Service (Amazon SQS) キューに接続できます。

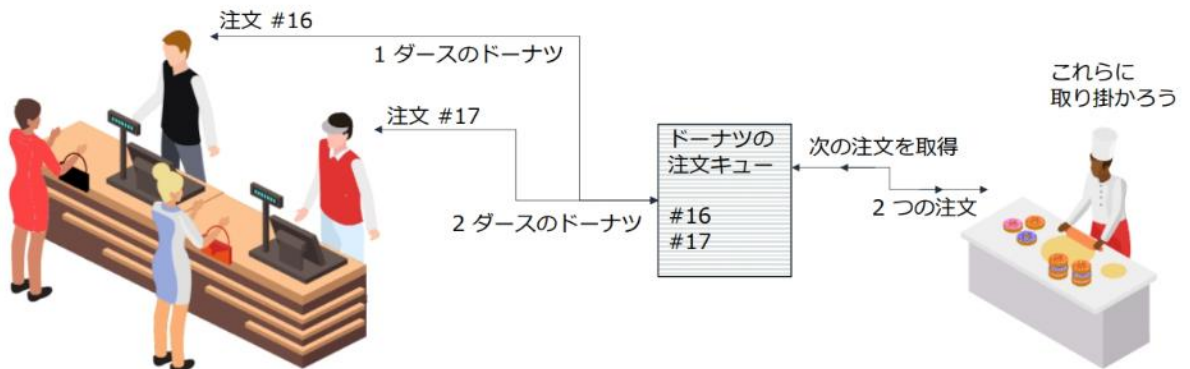
ストリーム処理

- メッセージのストリームは、コアエンティティです。多くのメッセージと一緒に確認することによってのみ、個々のメッセージの値を取得できます

- メッセージのストリームも、ほとんどの場合一定です。

- モバイルアプリからのクリックストリームデータ、アプリケーションログ、ホームセキュリティカメラフィードは、データを実用的にするのに必要な応答性がストリームによって得られる一般的な例です。

## ■ キューがプロデューサーと単一のコンシューマー間にバッファを提供する

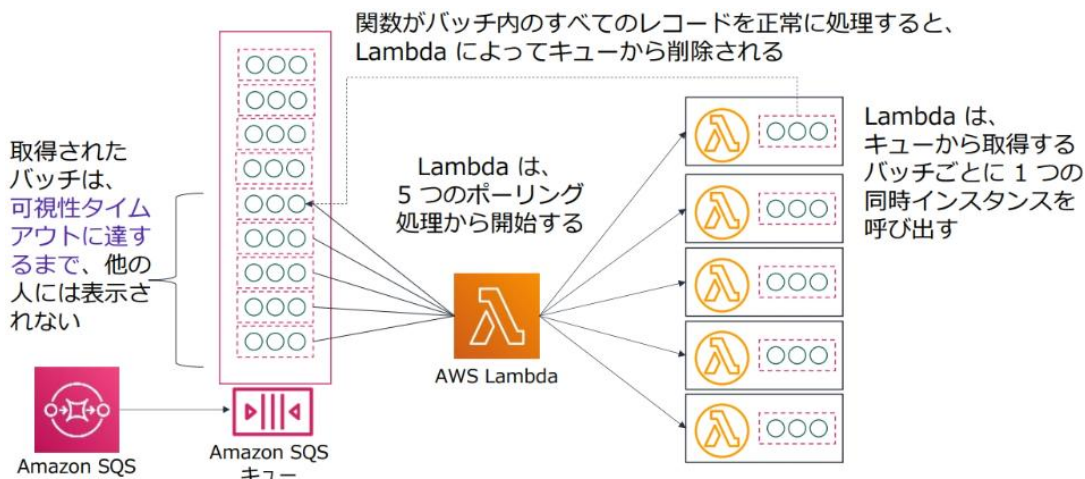


パン屋の例で、これはドーナツメーカーが作業する注文キューのようになります。ドーナツメーカーは、注文が到着するのに合わせてすばやく作業する必要はありません。キューによって、必要なペースで作業を実行できるバッファが提供されます。

顧客の観点からは、応答は依然として非同期であることに注目してください。

違いは、各アイテムが到着するたびに処理するのではなく、バッチで処理するためにバックエンドで一時的な保存が行われている点です。

## ■ Lambda が Amazon SQS キューをイベントソースとして管理する仕組み



Amazon SQS がイベントソースであり、別のサービスで SQS キューを設定した場合の動作は Lambda によって処理されます。Lambda は以下の内容を実行します。

- キューのポーリング
  - メッセージのバッチを含むイベントと同期して関数を呼び出す
  - 正常に処理されたレコードのバッチをキューから削除する
  - キューの深さとエラー率に基づいて、ポーラー数と Lambda の同時実行数をスケールアップまたはダウンする
- Lambda はデフォルトのポーラー数から開始し、各プロセスに対して 1 つのバッチを取得します。関数で設定したバッチサイズがキューに使用されます。
- Lambda は、ポーリング中にキューから取得したレコードの各バッチごとに、1 つの呼び出しを同期的に開始します。この例では、5 つのポーラーと 5 つの同時インスタンスの Lambda が示されています。

Lambda がレコードのバッチをプルすると、可視性タイムアウトの間、それらのレコードはキューの他のコンシューマー（つまり、他の Lambda 呼び出し）からは見えなくなります。可視性タイムアウトは、ユーザーが制御する設定です。また、Lambda がキューからバッチをプルするために使用するバッチサイズもユーザーが制御します。

Lambda は、メッセージのキューが大きくなっていることを確認すると、ポーリングプロセスの数を増やします。また、呼び出しから大量のエラーが返されると、ダウンストリームのコンシューマーに対する負荷が大きすぎると仮定して、プロセッサの数を減らします。

この動作の詳細については、「サーバーレスアプリケーションにおけるスケール処理」モジュールを参照してください。

Lambda がバッチを正常に処理すると、ユーザーに代わってメッセージをキューから削除するため、キューからメッセージを削除するためのコードを記述する必要はありません。



## Amazon SQS を Lambda のイベントソースとして扱う

処理内容	ベストプラクティス
<b>キューについて</b>	
キュータイプの選択 (標準または FIFO)	処理の順序が重要でない限り、標準キューを選択する
キューの可視性タイムアウトを設定する	可視性タイムアウトを関数タイムアウトの少なくとも 6 倍に設定する
キューに再処理ポリシーを設定する	デッドレターキューを設定し、最大受信数を選択してトラフィックのスパイクに対応するが、ボトルネックの頻度は制限する
<b>関数について</b>	
関数のバッチサイズ	高速のワークロードの場合はバッチサイズを大きく、長時間のワークロードの場合はバッチサイズを小さく設定する
関数タイムアウト	ほとんどの状況で関数が正常に完了できるように関数タイムアウトを設定する
部分的な障害	処理時に各レコードを削除するコードを含める
重複したメッセージ	べき等性の設計。同じレコードを 2 回処理しないようにする関数コードを含める

Lambda がキューのコンシューマーである場合、キューのいくつかの側面を処理します。Lambda がキューでどのように機能し、選択した内容がどのように機能するのかその仕組みを簡単に見てみましょう。

標準キューでは、順序は保証されず、メッセージが複数回配信されることがあります。しかし、1 秒あたりほぼ無制限にメッセージを処理できます。FIFO キューでは、順序はグループ ID ごとに厳密に保持されますが、スループットは制限されます。

このような高い可視性タイムアウトが必要になることは直感的に理解しづらいかもしれませんが、これは、適切なバランスを見つけるために、本番稼働の動作に基づいて設定をテスト、モニタリング、調整する必要がある理由の良い例です。

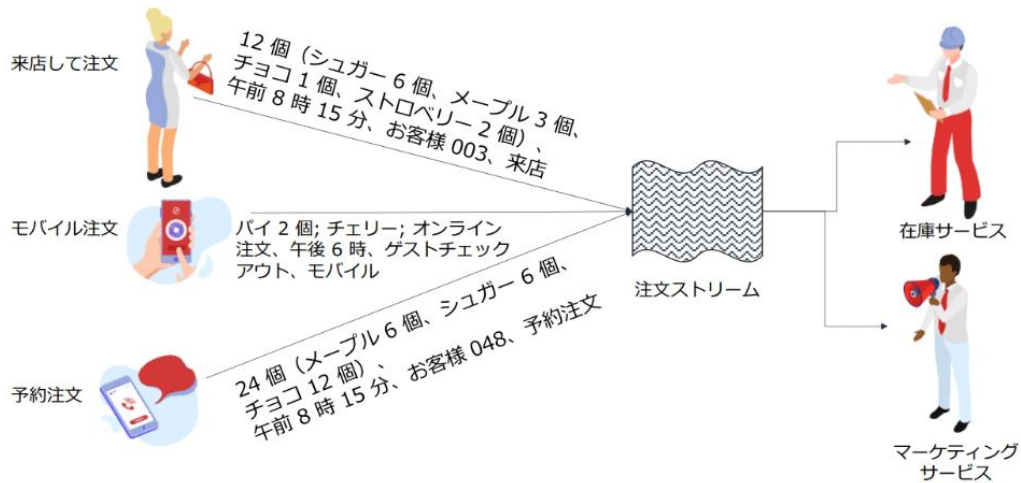
Lambda 関数がメッセージバッチを完了するうえで十分な時間を確保できるよう可視性タイムアウトを設定する必要があります。例えば、バッチサイズが 10 で、バッチの処理に 20 秒かかる Lambda 関数があるとした場合、20 秒を超える可視性タイムアウトが必要です。

また、可視性タイムアウトにバッファを残しておき、関数がスロットルされた場合に Lambda 呼び出しの再試行に対応できるようにする必要があります。これらのメッセージが処理される前に、可視性タイムアウトが終了しないようにします。

ベストプラクティスは、可視性タイムアウトを、関数に対して設定したタイムアウトの少なくとも 6 倍に設定することです。



## ■ 複数のコンシューマーに連続的に到着するストリームバッファデータ

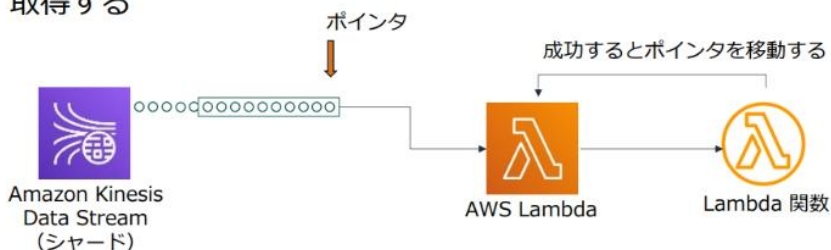


ストリームは、複数のソースから取得することができ、複数のコンシューマーを持つことができます。パン屋の例では、すべての注文に関するデータを収集するストリームを示しています。注文サービスまたは複数のサービスは、ストリームにデータを追加するプロデューサーです。在庫サービスは、ストリームのコンシューマーであり、その日に販売されなかった商品に対して割引を動的に提供するために使用している可能性があります。マーケティングサービスでは、このデータを活用してオンラインでショッピングをするユーザーにパーソナライズされたお勧め情報を提供する可能性もあります。ストリームを使用すると、一日の終わりにデータをバッチ処理するよりも、リアルタイムに近いデータを処理することができます。

## ■ Lambda がストリームをイベントソースとして管理する仕組み

Lambda はストリーム上の  
ポインタを保持し、バッチで  
各シャードからレコードを  
取得する

デフォルトでは Lambda は前の呼び出し  
が完了するまで、新しい呼び出しを試行  
しない



Lambda はストリームをポーリングし、メッセージのバッチを含むイベントを使用して関数を呼び出します。Lambda は、ストリーム上で最後に処理されたレコードのポインタを保持します。Lambda サービスは、シャードごとに 1 つのレコードのバッチを取得し、シャードごとに 1 つの Lambda 関数を呼び出します。そのバッチが正常に処理されると、Lambda はポインタ位置に基づいて、シャード内の次のレコードのバッチに移動します。

シャードの数を増やすか、シャードあたりの同時実行バッチを使用しない限り、Lambda が同時呼び出しを増やすことはありません。

ストリームのエラー処理を設定しない場合、失敗したレコードは、成功するまで、またはレコードが保持期間の終了に達するまで、シャードを完全にブロックします。

## ■ ストリームを Lambda のイベントソースとして扱う

処理内容	ベストプラクティス
ストリーム上	
Kinesis Data Streams: シャード数	
DynamoDB: テーブルの読み取り/書き込みスループットとスケーリング動作	
関数について	
- 関数のバッチサイズ   - シャードあたりの同時実行バッチ数   - バッチウィンドウ	- 速度を最適化するには、シャードあたりの同時バッチ数を増やす   - コストを最適化するには、バッチサイズを大きくするか、バッチウィンドウの設定を上げる
エラー処理オプション	BisectBatchOnError などの障害処理機能を使用する場合は、チェックポイント (コーディングが必要) を使用して重複を回避する
重複したメッセージ	冪等性の設計。 同じレコードを 2 回処理しないようにする関数コードを含める

## ■ DynamoDB と Kinesis Data Streams のスケーリング動作の区別

### Kinesis Data Streams

#### ポーリングレート

- Lambda は、各シャードを 1 秒に 1 回ポーリングする

#### シャード数

- ユーザーがストリームのシャード数を設定する

#### スケーリング

- シャード数は上下に変更でき、Auto Scaling を使用してプログラムで実行できる

### DynamoDB Streams

- Lambda は、各シャードを 1 秒に 4 回ポーリングする。

- DynamoDB テーブルの設定で、シャード数が自動的に決まる。

- テーブルは、テーブルの読み取り/書き込みボリュームを処理するためにシャードを増やすことがある。その場合、Lambda の同時実行数もそれに応じて増加する

前述の特徴は、DynamoDB と Kinesis Data Streams の両方に当てはまります。シャードの数を増やすと、実行中の Lambda の同時呼び出しの数が増加します。サービス構成によって、その数をどうするかを見分けることが重要です。

Kinesis Data Streams では、シャードの数と、その数が増えるタイミングを制御します。

DynamoDB ストリームでは、使用している DynamoDB テーブルに対する読み取り/書き込みキャパシティのプロビジョニング方法とスケーリングの判断によって、シャードの数が決まります。DynamoDB は、テーブルの設定方法とデータ量に基づいて、必要なシャードの数を自動的に調整します。

## 【演習2 EventBridge によるメッセージのファンアウト】

本演習では、社内で共有される技術記事や参考リンク（ブックマーク）を題材として、イベント駆動アーキテクチャの基本的な設計と実装を学習します。

この演習で扱う「記事」とは、ブログ記事やドキュメントを新しく作成するものではなく、**業務中に見つけた有用な Web ページ（URL）をチーム内で共有するナレッジ情報**を指します。

以下のような流れを想定しています。

- ・エンジニアが業務に役立つ技術記事や公式ドキュメントを見つける

↓

- ・その URL を社内アプリに登録する

↓

- ・個人用メモではなく「チームに共有する」操作を行う

この「**記事がチームに共有された**」という出来事をシステム上の **イベント** として扱います。

記事が共有された際、業務上は複数の処理が発生します。

- ・管理者や関係者へ共有通知を送りたい
- ・共有履歴としてログを残したい
- ・ナレッジ共有促進のため、キャンペーン（抽選）処理を実行したい

本演習では、これらの処理を **1つのプログラムにまとめて実装するのではなく**、イベントを起点として疎結合に分離する設計を採用します。

**Amazon DynamoDB** : 記事（ブックマーク）情報を保存するデータストア

**Amazon DynamoDB Streams** : 記事の追加・更新をイベントとして検知

**AWS Lambda** : DynamoDB の変更イベントを受け取り、イベントを整形して送信

**Amazon EventBridge** : 「記事が共有された」という事実を複数の処理に分配

**Amazon SNS** : 記事共有時に E メール通知を送信

**Amazon CloudWatch Logs** : イベント内容をログとして記録

### タスク 1 : DynamoDB テーブル作成

①DynamoDBコンソールで、「テーブル作成」を選択します。

②テーブル名「ArticlesTable」、パーティションキー「id」（文字列）を設定し、テーブル作成します。

③作成したテーブルの「エクスポートとストリーム」タブで「DynamoDB Stream」を「オンにする」に設定し有効化します。

「ビュータイプ」は「新旧イメージ」を選択します。

### タスク 2 : Lambda関数 作成①（EventBridge用）

DynamoDBテーブルに新しい記事が追加されたとき、その記事が共有される場合だけ（shared=true）、EventBridgeにイベントを送信する、関数を作成します。

①**Lambda**コンソール画面を表示し、**関数の作成**を実行します。

②関数名は「StreamToEventBridge」とし、ランタイムは「**Python 3.12**」を選択します。

③実行ロールには基本的なLambda関数の権限「AWSLambdaBasicExecutionRole」を付与し、ログをCloudWatchに出力できるようにします。

さらに、EventBridgeにイベントを送る権限も必要です。

イベントソースに「EventBridge」を選択し、必要なアクションには「events:PutEvents」を選択、今回は演習のため、対象リソースは「全て\*」を選択します。

またLambdaがDynamoDB Streamを読むための権限「AWSLambdaDynamoDBExecutionRole」も追加します。

④関数のコードは以下に置き換え、デプロイも忘れずに実行します。

```
import json
import boto3

eventbridge = boto3.client("events")
def lambda_handler(event, context):
 for record in event["Records"]:
 if record["eventName"] != "INSERT":
 continue
 new_image = record["dynamodb"]["NewImage"]
 shared = new_image.get("shared", {}).get("BOOL", False)
 if not shared:
 print("shared is false or missing. skip event")
 return
 detail = {
 "id": new_image["id"]["S"],
 "title": new_image["title"]["S"],
 "content": new_image["content"]["S"],
 "shared": shared
 }
 response = eventbridge.put_events(
 Entries=[
 {
 "Source": "articles.service",
 "DetailType": "ArticleShared",
 "Detail": json.dumps(detail),
 "EventBusName": "bookmarks-bus"
 }
]
)
 print("EventBridge response:", response)
```

## タスク 3： LambdaとDynamoDB Streamの接続

DynamoDBのテーブルに新規作成があれば（記事が投稿されたら）Lambda関数が実行されるようにトリガーの設定をします。

- ①作成済Lambda関数の コンソール画面で「設定」タブを選択します。
- ②「トリガー」タブを選択し、「トリガーを追加」ボタンをクリックします。
- ③ソースで「DynamoDB」を選択し、「BookmarksTable」を選びます。
- ④バッチサイズを「5」に設定し追加します。

## タスク 4： EventBridge イベントバス作成

イベントの通り道であるEventBridge イベントバスを作成します。

- ①Amazon EventBridgeコンソールを開き、画面左メニューから「イベントバス」を選択します。
- ②「イベントバスを作成」ボタンをクリックし、名前に「bookmarks-bus」と入力し作成します。

## タスク 5: 通知用 SNS トピック 作成

新しい投稿があった場合、メール送信で通知をするための通知先を設定します。

- ① SNSコンソールを表示し、画面左のメニューから「トピック」を選択します。
- ② 「トピックの作成」ボタンをクリックし、以下の設定で作成します。
  - ・タイプ：スタンダード
  - ・名前：BookmarkNotification
- ③ 続いて「サブスクリプションの作成」ボタンをクリックし、プロトコルから「Eメール」を選択します。
- ④ エンドポイントには通知を受信できるメールアドレスを入力します。  
(動作確認のため、ダミーアドレスではなくメール受信できる有効なメールアドレスの入力を推奨)
- ⑤ 設定したメールアドレス宛に確認メール (Subscription Confirmation) が届いているため、メール本文にある「Confirm subscription」リンクをクリックし、承認します。
- ⑥ コンソール画面のサブスクリプション欄にある自身のメールアドレスが「確認済み」のステータスに更新されたことを確認します。

## タスク 6: EventBridge ルール作成

新しい投稿での共有 (shared) がオンの場合のみ通知メールが送信されるようルールを作成します。

- ① 「EventBridge」コンソールを表示し、画面左の「ルール」を選択します。
- ② 「イベントバスの選択」欄は作成済の「bookmarks-bus」に切り替え、「ルールの作成」ボタンをクリックします。
- ③ 名前には「notify-rule」と入力し、イベントバスは「bookmarks-bus」が選択されていることを確認し「次へ」をクリックします。
- ④ 「イベントソース」は「その他」を選択し、「イベントパターン」は「カスタム」をクリックします。
- ⑤ JSONエディタに以下を入力します。

```
{
 "source": ["articles.service"],
 "detail-type": ["ArticleShared"]
}
```

- ⑥ ターゲットには SNS: BookmarkNotification を選択し、作成します。

## タスク 7: Lambda関数 作成② (DynamoDB書き込み用)

DynamoDBに投稿を書き込む関数を作成します。

①Lambdaのコンソールを表示し「関数の作成」を選択します。

②関数名: 「DynamoDBWriteFunction」

ランタイム: Python 3.12

実行ロール: 基本的な Lambda 権限 を設定します。

③さらにDynamoDBに書き込むための以下の権限を追加します。「AmazonDynamoDBFullAccess」

④関数のコードは以下の通りです。デプロイも実行します。

```
import json
import uuid
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("ArticlesTable")

def lambda_handler(event, context):
 body = json.loads(event["body"])

 item = {
 "id": str(uuid.uuid4()), # ← 必須
 "title": body["title"],
 "content": body["content"],
 "shared": body["shared"] # ← 必須 (bool)
 }

 table.put_item(Item=item)

 return {
 "statusCode": 200,
 "headers": {
 "Access-Control-Allow-Origin": "*",
 "Access-Control-Allow-Headers": "Content-
Type",
 "Access-Control-Allow-Methods":
"OPTIONS,POST"
 },
 "body": json.dumps({
 "message": "Article created successfully!"
 })
 }
```

## タスク 8: API Gateway 作成

ブラウザからLambda関数を呼び出すためのAPI Gatewayを作成します。

①API Gatewayのコンソールを表示し「**HTTP API**」を構築します。

②任意のAPI名を付け、「統合を追加」を選択し、統合先: Lambda、  
対象 Lambda: DynamoDBWriteFunction (前タスクで作成済関数) を設定します。

③「ルートを追加」を選択し、メソッド: POST、パス: /articles、を設定します。

④作成したAPIで、CORS設定をします。CORSを 有効化、Allow-Origin: \*、Allow-Headers: content-type  
Allow-Methods: POST, OPTIONSを選択し、保存します。

⑤API Gateway画面の左側「Stages (ステージ)」に呼び出しURLが表示されていることを確認します。  
(次のタスクで必要です)



## タスク 9: フロントエンド (UI) 作成

新しい投稿をするためにUI (Web画面) を作成し、簡易的なローカルサーバーを立ち上げ動作確認します。

①開発環境のエディタで以下のHTMLを作成し、任意の名前で保存します。

```
<!DOCTYPE html>
<html lang="ja">
<head>
 <meta charset="UTF-8">
 <title>Knowledge Share App</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 margin: 40px;
 }
 label {
 display: block;
 margin-top: 15px;
 }
 button {
 margin-top: 20px;
 padding: 10px 20px;
 }
 #result {
 margin-top: 20px;
 font-weight: bold;
 }
 </style>
</head>
<body>

 <h1>ナレッジ投稿</h1>

 <form id="articleForm">
 <label>
 タイトル:
 <input type="text" id="title" required>
 </label>

 <label>
 内容:
 <textarea id="content" rows="5"
required></textarea>
 </label>

 <label>
 <input type="checkbox" id="shared">
 ナレッジを共有する (Shared)
 </label>

 <button type="submit">Submit</button>
 </form>

 <div id="result"></div>
```

このURLは作成済APIGatewayの呼び出しURLとなります。  
(呼び出しURL末尾にパス「/articles」を追加)

```
<script>
 const form =
document.getElementById("articleForm");
 const resultDiv =
document.getElementById("result");

 form.addEventListener("submit", async (event) =>
{
 event.preventDefault();

 const title =
document.getElementById("title").value;
 const content =
document.getElementById("content").value;
 const shared =
document.getElementById("shared").checked;

 const payload = {
 title: title,
 content: content,
 shared: shared
 };

 console.log("send payload:", payload);

 try {
 const response = await fetch(
 "https://xxxxx.execute-api.ap-northeast-
1.amazonaws.com/articles",
 {
 method: "POST",
 headers: {
 "Content-Type": "application/json"
 },
 body: JSON.stringify(payload)
 }
);

 console.log("status:", response.status);

 if (!response.ok) {
 throw new Error(`HTTP error
${response.status}`);
 }

 const data = await response.json();
 console.log("response body:", data);

 resultDiv.textContent = "Article created
successfully!";
 } catch (error) {
 console.error("fetch error:", error);
 resultDiv.textContent = "エラーが発生しました。
コンソールを確認してください。";
 }
 });
</script>

</body>
</html>
```

## タスク 10: 動作確認

全体の動作確認のため、簡易Webサーバーを起動し、新規記事を投稿します。DynamoDBに項目が追加され、共有（Shared）オンの場合のみ自身のメールアドレスに通知メールが届くことを確認します。

①ローカル簡易 Web サーバーを起動するため、コマンドプロンプトなどで以下コマンドを実行します。

```
python -m http.server 8080
```

②ブラウザで以下URLにアクセスします。

<http://localhost:8080/HTMLファイル名.html>

③表示された画面で、内容を入力し、ナレッジ共有をオンにし「Submit」ボタンを押下します。

④画面上に「 **Article created successfully!** 」と表示されていれば成功です。  
DynamoDBコンソールで項目が追加されていることを確認します。  
さらに自身のメールアドレスに通知メールが届いていることを確認します。

## 【オプション演習】 1: 抽選機能追加

ナレッジ共有促進化のため、共有設定をオンにして投稿すると抽選が実行できる機能を追加します。

①抽選用Lambda関数を作成します。

関数名: LotteryFunction

ランタイム: Python 3.12

実行ロール: 既存の基本 Lambda ロール

②コードは以下の通りです。

```
import json
import random

def lambda_handler(event, context):
 detail = event["detail"]

 # 30%で当選
 win = random.random() < 0.3

 result = {
 "articleId": detail["id"],
 "winner": win
 }

 print("Lottery result:", result)

 return {
 "statusCode": 200,
 "body": json.dumps(result)
 }
```

③EventBridge に抽選ルールを追加します。ルール作成画面で以下の通り設定します。

名前: lottery-rule

イベントバス: bookmarks-bus

ルールタイプ: イベントパターン

```
{
 "detail-type": ["ArticleShared"],
 "detail": {
 "shared": [true]
 }
}
```

ターゲットタイプ: Lambda

関数: LotteryFunction

④抽選結果取得用 API を作成します。既存のAPI Gateway画面を表示し、**GET /lottery-result** を追加します。

統合先: LotteryFunctionを設定します。

⑤新規HTMLファイルを作成し、以下を入力します。

```
<!DOCTYPE html>
<html>
<body>
 <h2>Create Article</h2>

 <input id="title" placeholder="title">

 <textarea id="content"></textarea>

 <label>
 <input type="checkbox" id="shared">
 Share this article
 </label>

 <button onclick="send()">Submit</button>
```

⑤新規HTMLファイルを作成し、以下を入力します。

```
<!DOCTYPE html>
<html>
<body>
 <h2>Create Article</h2>

 <input id="title" placeholder="title">

 <textarea id="content"></textarea>

 <label>
 <input type="checkbox" id="shared">
 Share this article
 </label>

 <button onclick="send()">Submit</button>

 <h3 id="result"></h3>

<script>
async function send() {
 const res = await fetch(
 "https://XXXX.execute-api.ap-northeast-1.amazonaws.com/articles",
 {
 method: "POST",
 headers: {"Content-Type": "application/json"},
 body: JSON.stringify({
 title: title.value,
 content: content.value,
 shared: shared.checked
 })
 }
);

 if (!res.ok) {
 document.getElementById("result").innerText = "投稿に失敗しました";
 return;
 }

 // 抽選結果取得
 const lottery = await fetch(
 "https://XXXX.execute-api.ap-northeast-1.amazonaws.com/lottery-result"
).then(r => r.json());

 document.getElementById("result").innerText =
 lottery.winner
 ? "You won. Congratulations!"
 : "Unfortunately, you were not selected.";
}
</script>
</body>
</html>
```

⑥簡易Webサーバーが起動している状態で⑤ファイルを開き、動作確認します。抽選結果が画面に表示されることを確認します。